

Physlib Monthly Report

1–30 June 2024

Contributors: Joseph Tooby-Smith, Pietro Monticone

Generated 14 August 2026 from commit `0116994`.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–30 June 2024 on branch `master` of [leanprover-community/physlib](#), between commits `801a929` and `0116994`. Over that period 2 contributors landed 147 commits, changing 5,083 lines across 99 files and adding 444 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	2
2	Details by file	3
2.1	HepLean.AnomalyCancellation.Basic	3
2.2	HepLean.AnomalyCancellation.MSSMNu.OrthogY3B3.ToSols	3
2.3	HepLean.AnomalyCancellation.MSSMNu.Permutations	5
2.4	HepLean.AnomalyCancellation.PureU1.ConstAbs	5
2.5	HepLean.AnomalyCancellation.PureU1.Even.LineInCubic	8
2.6	HepLean.AnomalyCancellation.PureU1.Even.Parameterization	9
2.7	HepLean.AnomalyCancellation.PureU1.LineInPlaneCond	9
2.8	HepLean.AnomalyCancellation.PureU1.Odd.LineInCubic	9
2.9	HepLean.AnomalyCancellation.PureU1.Odd.Parameterization	10
2.10	HepLean.AnomalyCancellation.PureU1.Permutations	10
2.11	HepLean.AnomalyCancellation.PureU1.Sort	10
2.12	HepLean.AnomalyCancellation.PureU1.VectorLike	11
2.13	HepLean.AnomalyCancellation.SM.Permutations	11
2.14	HepLean.AnomalyCancellation.SMNu.Permutations	11
2.15	HepLean.AnomalyCancellation.SMNu.PlusU1.BoundPlaneDim	11
2.16	HepLean.BeyondTheStandardModel.TwoHDM.Basic	12
2.17	HepLean.FeynmanDiagrams.Basic	12
2.18	HepLean.FeynmanDiagrams.Instances.ComplexScalar	24
2.19	HepLean.FeynmanDiagrams.Instances.Phi4	24
2.20	HepLean.FeynmanDiagrams.Momentum	25
2.21	HepLean.FlavorPhysics.CKMMatrix.Basic	28

2.22	HepLean.FlavorPhysics.CKMMatrix.PhaseFreedom	28
2.23	HepLean.SpaceTime.AsSelfAdjointMatrix	28
2.24	HepLean.SpaceTime.Basic	29
2.25	HepLean.SpaceTime.LorentzAlgebra.Basic	31
2.26	HepLean.SpaceTime.LorentzAlgebra.Basis	32
2.27	HepLean.SpaceTime.LorentzGroup.Basic	33
2.28	HepLean.SpaceTime.LorentzGroup.Boosts	36
2.29	HepLean.SpaceTime.LorentzGroup.Orthochronous	37
2.30	HepLean.SpaceTime.LorentzGroup.Proper	40
2.31	HepLean.SpaceTime.LorentzGroup.Rotations	41
2.32	HepLean.SpaceTime.Metric	42
2.33	HepLean.SpaceTime.SL2C.Basic	45
2.34	HepLean.StandardModel.Basic	46
2.35	HepLean.StandardModel.HiggsBoson.Basic	46
2.36	HepLean.StandardModel.HiggsBoson.TargetSpace	49
2.37	scripts.check_file_imports	53
2.38	scripts.type_former_lint	54

1 Introduction

During Jun 2024, 2 contributors submitted 147 commits that changed 5,083 lines (+3,513 / -1,570) across 99 files spanning 18 top-level areas of the repository. Of those, 20 files were newly added and 5 were removed outright. This report catalogues 444 newly added Lean declarations: 185 lemmas, 167 defs, 79 instances, 5 abbrevs, 3 theorems, 3 structures, and 2 class.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Joseph Tooby-Smith	8,692	128
Pietro Monticone	529	19

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

No reviewers identified for this month.

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
HepLean/AnomalyCancellation	44	—	—	+572	-669
HepLean/FeynmanDiagrams	4	+4	—	+1,146	-0
HepLean/SpaceTime	13	+3	—	+709	-306
docs	8	+8	—	+495	-0
HepLean/StandardModel	4	—	—	+161	-240
scripts	6	+2	-3	+146	-105
HepLean/FlavorPhysics	6	—	—	+86	-125
.github/workflows	3	—	-1	+27	-34
README.md	1	—	—	+15	-30
docs/_layouts	1	+1	—	+42	-0
HepLean/BeyondTheStandardModel	1	+1	—	+37	-0
HepLean/Mathematics	2	—	—	+10	-21
lake-manifest.json	1	—	—	+11	-20
lakefile.toml	1	—	—	+24	-0
docs/assets	1	+1	—	+21	-0
lakefile.lean.bak	1	—	-1	+0	-17
HepLean.lean	1	—	—	+10	-2
lean-toolchain	1	—	—	+1	-1

1.4 New declarations by kind

Kind	Count
------	-------

lemma	185
def	167
instance	79
abbrev	5
theorem	3
structure	3
class	2

2 Details by file

2.1 HepLean.AnomalyCancellation.Basic

[HepLean/AnomalyCancellation/Basic.lean](#) on GitHub.

def Charges

[\[source\]](#)

The charges as functions from $\text{Fin } \chi.\text{numberCharges} \rightarrow \mathbb{Q}$.

```
def Charges (χ : ACCSystemCharges) : Type
```

instance Module.Finite $\mathbb{Q}$ χ .Charges

[\[source\]](#)

```
instance (χ : ACCSystemCharges) : Module.Finite  $\mathbb{Q}$   $\chi$ .Charges
```

def IsSolution

[\[source\]](#)

We say a charge S is a solution if it extends to a solution.

```
def IsSolution (χ : ACCSystem) (S :  $\chi$ .Charges) : Prop
```

2.2 HepLean.AnomalyCancellation.MSSMNu.OrthogY3B3.ToSols

[HepLean/AnomalyCancellation/MSSMNu/OrthogY3B3/ToSols.lean](#) on GitHub.

def LineEqProp

[\[source\]](#)

A condition for the quad line in the plane spanned by R , Y_3 and B_3 to sit in the cubic, and for the cube line to sit in the quad.

```
def LineEqProp (R : MSSMACC.AnomalyFreePerp) : Prop
```

instance Decidable (LineEqProp R)

[\[source\]](#)

```
instance (R : MSSMACC.AnomalyFreePerp) : Decidable (LineEqProp R)
```

def LineEqPropSol

[\[source\]](#)

A condition on Sols which we will show in `lineEqPropSol_iff_proj_lineEqProp` that is equivalent to the condition that the `proj` of the solution satisfies `lineEqProp`.

```
def LineEqPropSol (R : MSSMACC.Sols) : Prop
```

def InQuadProp[\[source\]](#)

A condition which is satisfied if the plane spanned by R , Y_3 and B_3 lies entirely in the quadratic surface.

```
def InQuadProp (R : MSSMACC.AnomalyFreePerp) : Prop
```

instance Decidable (InQuadProp R)[\[source\]](#)

```
instance (R : MSSMACC.AnomalyFreePerp) : Decidable (InQuadProp R)
```

def InQuadSolProp[\[source\]](#)

A condition which is satisfied if the plane spanned by the solutions R , Y_3 and B_3 lies entirely in the quadratic surface.

```
def InQuadSolProp (R : MSSMACC.Sols) : Prop
```

def InCubeProp[\[source\]](#)

A condition which is satisfied if the plane spanned by R , Y_3 and B_3 lies entirely in the cubic surface.

```
def InCubeProp (R : MSSMACC.AnomalyFreePerp) : Prop
```

instance Decidable (InCubeProp R)[\[source\]](#)

```
instance (R : MSSMACC.AnomalyFreePerp) : Decidable (InCubeProp R)
```

def InCubeSolProp[\[source\]](#)

A condition which is satisfied if the plane spanned by the solutions R , Y_3 and B_3 lies entirely in the cubic surface.

```
def InCubeSolProp (R : MSSMACC.Sols) : Prop
```

def InLineEq[\[source\]](#)

Those charge assignments perpendicular to Y_3 and B_3 which satisfy the condition `lineEqProp`.

```
def InLineEq : Type
```

def InQuad[\[source\]](#)

Those charge assignments perpendicular to Y_3 and B_3 which satisfy the condition `lineEqProp` and `inQuadProp`.

```
def InQuad : Type
```

def InQuadCube[\[source\]](#)

Those charge assignments perpendicular to Y_3 and B_3 which satisfy the condition `lineEqProp`, `inQuadProp` and `inCubeProp`.

```
def InQuadCube : Type
```

def NotInLineEqSol[\[source\]](#)

Those solutions which do not satisfy the condition `lineEqPropSol`.

```
def NotInLineEqSol : Type
```

def InLineEqSol[\[source\]](#)

Those solutions which satisfy the condition `lineEqPropSol` by not `inQuadSolProp`.

```
def InLineEqSol : Type
```

def InQuadSol[\[source\]](#)

Those solutions which satisfy the condition `lineEqPropSol` and `inQuadSolProp` but not `inCubeSolProp`.

```
def InQuadSol : Type
```

def InQuadCubeSol[\[source\]](#)

Those solutions which satisfy the condition all the conditions `lineEqPropSol`, `inQuadSolProp` and `inCubeSolProp`.

```
def InQuadCubeSol : Type
```

2.3 HepLean.AnomalyCancellation.MSSMNu.Permutations

[HepLean/AnomalyCancellation/MSSMNu/Permutations.lean](#) on GitHub.

def PermGroup[\[source\]](#)

The group of family permutations is S_3 ⁶

```
@[simp]
```

```
def PermGroup
```

instance Group PermGroup[\[source\]](#)

```
@[simp]
```

```
instance : Group PermGroup
```

2.4 HepLean.AnomalyCancellation.PureU1.ConstAbs

[HepLean/AnomalyCancellation/PureU1/ConstAbs.lean](#) on GitHub.

def ConstAbsProp[\[source\]](#)

The condition for two rationals to have the same square (equivalent to same abs).

```
def ConstAbsProp :  $\mathbb{Q} \times \mathbb{Q} \rightarrow \text{Prop}$ 
```

def ConstAbs[\[source\]](#)

The condition on a charge assignment S to have constant absolute value among charges.

```
@[simp]
```

```
def ConstAbs (S : (PureU1 n).Charges) : Prop
```

def ConstAbsSorted[\[source\]](#)

The condition for a set of charges to be sorted, and have constAbs

```
def ConstAbsSorted (S : (PureU1 n).Charges) : Prop
```

lemma lt_eq[\[source\]](#)

```
lemma lt_eq {k i : Fin n.succ} (hk : S k ≤ 0) (hik : i ≤ k) : S i = S k
```

lemma val_le_zero[\[source\]](#)

```
lemma val_le_zero {i : Fin n.succ} (hi : S i ≤ 0) : S i = S (0 : Fin n.succ)
```

lemma gt_eq[\[source\]](#)

```
lemma gt_eq {k i : Fin n.succ} (hk : 0 ≤ S k) (hik : k ≤ i) : S i = S k
```

lemma zero_gt[\[source\]](#)

```
lemma zero_gt (h0 : 0 ≤ S (0 : Fin n.succ)) (i : Fin n.succ) : S (0 : Fin n.succ) = S i
```

lemma opposite_signs_eq_neg[\[source\]](#)

```
lemma opposite_signs_eq_neg {i j : Fin n.succ} (hi : S i ≤ 0) (hj : 0 ≤ S j) : S i = - S j
```

lemma is_zero[\[source\]](#)

```
lemma is_zero (h0 : S (0 : Fin n.succ) = 0) : S = 0
```

def Boundary[\[source\]](#)

A boundary of $S : (\text{PureU1 } n.\text{succ}).\text{charges}$ (assumed sorted, constAbs and non-zero) is defined as a element of $k \in \text{Fin } n$ such that $S \ k.\text{castSucc}$ and $S \ k.\text{succ}$ are different signs.

```
@[simp]
```

```
def Boundary (S : (PureU1 n.succ).Charges) (k : Fin n) : Prop
```

lemma boundary_castSucc[\[source\]](#)

```
lemma boundary_castSucc {k : Fin n} (hk : Boundary S k) : S k.castSucc = S (0 : Fin n.succ)
```

lemma boundary_succ[\[source\]](#)

```
lemma boundary_succ {k : Fin n} (hk : Boundary S k) : S k.succ = - S (0 : Fin n.succ)
```

lemma boundary_split[\[source\]](#)

```
lemma boundary_split (k : Fin n) : k.succ.val + (n.succ - k.succ.val) = n.succ
```

lemma boundary_accGrav'[\[source\]](#)

```
lemma boundary_accGrav' (k : Fin n) : accGrav n.succ S =
  ∑ i : Fin (k.succ.val + (n.succ - k.succ.val)), S (Fin.cast (boundary_split k) i)
```

lemma boundary_accGrav''[\[source\]](#)

```
lemma boundary_accGrav'' (k : Fin n) (hk : Boundary S k) :
  accGrav n.succ S = (2 * ↑k + 1 - ↑n) * S (0 : Fin n.succ)
```

def HasBoundary[\[source\]](#)

We say a $S \in \text{charges}$ has a boundary if there exists a $k \in \text{Fin } n$ which is a boundary.

```
@[simp]
```

```
def HasBoundary (S : (PureU1 n.succ).Charges) : Prop
```

lemma not_hasBoundary_zero_le[\[source\]](#)

```
lemma not_hasBoundary_zero_le (hnot : ¬ (HasBoundary S)) (h0 : S (0 : Fin n.succ) < 0) :
  ∀ i, S (0 : Fin n.succ) = S i
```

lemma not_hasBoundary_zero[\[source\]](#)

```
lemma not_hasBoundary_zero (hnot : ¬ (HasBoundary S)) (i : Fin n.succ) :
  S (0 : Fin n.succ) = S i
```

lemma not_hasBoundary_grav[\[source\]](#)

```
lemma not_hasBoundary_grav (hnot : ¬ (HasBoundary S)) :
  accGrav n.succ S = n.succ * S (0 : Fin n.succ)
```

lemma AFL_hasBoundary[\[source\]](#)

```
lemma AFL_hasBoundary (h : A.val (0 : Fin n.succ) ≠ 0) : HasBoundary A.val
```


lemma AFL_odd_noBoundary[\[source\]](#)

```
lemma AFL_odd_noBoundary {A : (PureU1 (2 * n + 1)).LinSols} (h : ConstAbsSorted A.val)
  (hA : A.val (0 : Fin (2*n + 1)) ≠ 0) : ¬ HasBoundary A.val
```

lemma AFL_odd_zero[\[source\]](#)

```
lemma AFL_odd_zero {A : (PureU1 (2 * n + 1)).LinSols} (h : ConstAbsSorted A.val) :
  A.val (0 : Fin (2 * n + 1)) = 0
```

theorem AFL_odd[\[source\]](#)

```
theorem AFL_odd (A : (PureU1 (2 * n + 1)).LinSols) (h : ConstAbsSorted A.val) :
  A = 0
```

lemma AFL_even_Boundary[\[source\]](#)

```
lemma AFL_even_Boundary {A : (PureU1 (2 * n.succ)).LinSols} (h : ConstAbsSorted A.val)
  (hA : A.val (0 : Fin (2 * n.succ)) ≠ 0) {k : Fin (2 * n + 1)} (hk : Boundary A.val k) :
  k.val = n
```

lemma AFL_even_below'[\[source\]](#)

```
lemma AFL_even_below' {A : (PureU1 (2 * n.succ)).LinSols} (h : ConstAbsSorted A.val)
  (hA : A.val (0 : Fin (2 * n.succ)) ≠ 0) (i : Fin n.succ) :
  A.val (Fin.cast (split_equal n.succ) (Fin.castAdd n.succ i)) = A.val (0 : Fin (2*n.succ))
```

lemma AFL_even_below[\[source\]](#)

```
lemma AFL_even_below (A : (PureU1 (2 * n.succ)).LinSols) (h : ConstAbsSorted A.val)
  (i : Fin n.succ) :
  A.val (Fin.cast (split_equal n.succ) (Fin.castAdd n.succ i))
  = A.val (0 : Fin (2*n.succ))
```

lemma AFL_even_above'[\[source\]](#)

```
lemma AFL_even_above' {A : (PureU1 (2 * n.succ)).LinSols} (h : ConstAbsSorted A.val)
  (hA : A.val (0 : Fin (2*n.succ)) ≠ 0) (i : Fin n.succ) :
  A.val (Fin.cast (split_equal n.succ) (Fin.natAdd n.succ i)) =
  - A.val (0 : Fin (2*n.succ))
```

lemma AFL_even_above[\[source\]](#)

```
lemma AFL_even_above (A : (PureU1 (2 * n.succ)).LinSols) (h : ConstAbsSorted A.val)
  (i : Fin n.succ) :
  A.val (Fin.cast (split_equal n.succ) (Fin.natAdd n.succ i)) =
  - A.val (0 : Fin (2*n.succ))
```

2.5 HepLean.AnomalyCancellation.PureU1.Even.LineInCubic

[HepLean/AnomalyCancellation/PureU1/Even/LineInCubic.lean](#) on GitHub.

def LineInCubic[\[source\]](#)

A property on `LinSols`, satisfied if every point on the line between the two planes in the basis through that point is in the cubic.

```
def LineInCubic (S : (PureU1 (2 * n.succ)).LinSols) : Prop
```

def LineInCubicPerm[\[source\]](#)

We say a `LinSol` satisfies `lineInCubicPerm` if all its permutations satisfy `lineInCubic`.

```
def LineInCubicPerm (S : (PureU1 (2 * n.succ)).LinSols) : Prop
```

2.6 HepLean.AnomalyCancellation.PureU1.Even.Parameterization

[HepLean/AnomalyCancellation/PureU1/Even/Parameterization.lean](#) on GitHub.

def GenericCase[\[source\]](#)

A proposition on a solution which is true if `accCubeTriLinSymm (P g, P g, P! f) ≠ 0`. In this case our parameterization above will be able to recover this point.

```
def GenericCase (S : (PureU1 (2 * n.succ)).Sols) : Prop
```

def SpecialCase[\[source\]](#)

A proposition on a solution which is true if `accCubeTriLinSymm (P g, P g, P! f) = 0`.

```
def SpecialCase (S : (PureU1 (2 * n.succ)).Sols) : Prop
```

2.7 HepLean.AnomalyCancellation.PureU1.LineInPlaneCond

[HepLean/AnomalyCancellation/PureU1/LineInPlaneCond.lean](#) on GitHub.

def LineInPlaneProp[\[source\]](#)

The proposition on three rationals to satisfy the `linInPlane` condition.

```
def LineInPlaneProp : ℚ × ℚ × ℚ → Prop
```

def LineInPlaneCond[\[source\]](#)

The proposition on a `LinSol` to satisfy the `linInPlane` condition.

```
def LineInPlaneCond (S : (PureU1 (n)).LinSols) : Prop
```

2.8 HepLean.AnomalyCancellation.PureU1.Odd.LineInCubic

[HepLean/AnomalyCancellation/PureU1/Odd/LineInCubic.lean](#) on GitHub.

def LineInCubic[\[source\]](#)

A property on `LinSols`, satisfied if every point on the line between the two planes in the basis through that point is in the cubic.

```
def LineInCubic (S : (PureU1 (2 * n + 1)).LinSols) : Prop
```

def LineInCubicPerm[\[source\]](#)

We say a `LinSol` satisfies `lineInCubicPerm` if all its permutations satisfy `lineInCubic`.

```
def LineInCubicPerm (S : (PureU1 (2 * n + 1)).LinSols) : Prop
```

2.9 HepLean.AnomalyCancellation.PureU1.Odd.Parameterization

[HepLean/AnomalyCancellation/PureU1/Odd/Parameterization.lean](#) on GitHub.

def GenericCase[\[source\]](#)

A proposition on a solution which is true if `accCubeTriLinSymm (P g, P g, P! f) ≠ 0`. In this case our parameterization above will be able to recover this point.

```
def GenericCase (S : (PureU1 (2 * n.succ + 1)).Sols) : Prop
```

def SpecialCase[\[source\]](#)

A proposition on a solution which is true if `accCubeTriLinSymm (P g, P g, P! f) ≠ 0`. In this case we will show that `S` is zero if it is true for all permutations.

```
def SpecialCase (S : (PureU1 (2 * n.succ + 1)).Sols) : Prop
```

2.10 HepLean.AnomalyCancellation.PureU1.Permutations

[HepLean/AnomalyCancellation/PureU1/Permutations.lean](#) on GitHub.

def PermGroup[\[source\]](#)

The permutation group of the n -fermions.

```
@[simp]
```

```
def PermGroup (n : ℕ)
```

instance Group (PermGroup n)[\[source\]](#)

```
instance {n : ℕ} : Group (PermGroup n)
```

2.11 HepLean.AnomalyCancellation.PureU1.Sort

[HepLean/AnomalyCancellation/PureU1/Sort.lean](#) on GitHub.

def Sorted[\[source\]](#)

We say a charge is sorted if for all $i \leq j$, then $S_i \leq S_j$.

```
@[simp]
def Sorted {n : ℕ} (S : (PureU1 n).Charges) : Prop
```

2.12 HepLean.AnomalyCancellation.PureU1.VectorLike

[HepLean/AnomalyCancellation/PureU1/VectorLike.lean](#) on GitHub.

def VectorLikeEven [\[source\]](#)

A charge configuration for n even is vector like if when sorted the i th element is equal to the negative of the $n + i$ th element.

```
@[simp]
def VectorLikeEven (S : (PureU1 (2 * n)).Charges) : Prop
```

2.13 HepLean.AnomalyCancellation.SM.Permutations

[HepLean/AnomalyCancellation/SM/Permutations.lean](#) on GitHub.

def PermGroup [\[source\]](#)

The group of S_n permutations for each species.

```
@[simp]
def PermGroup (n : ℕ)
```

instance Group (PermGroup n) [\[source\]](#)

```
@[simp]
instance : Group (PermGroup n)
```

2.14 HepLean.AnomalyCancellation.SMNu.Permutations

[HepLean/AnomalyCancellation/SMNu/Permutations.lean](#) on GitHub.

def PermGroup [\[source\]](#)

The group of S_n permutations for each species.

```
@[simp]
def PermGroup (n : ℕ)
```

instance Group (PermGroup n) [\[source\]](#)

```
@[simp]
instance : Group (PermGroup n)
```

2.15 HepLean.AnomalyCancellation.SMNu.PlusU1.BoundPlaneDim

[HepLean/AnomalyCancellation/SMNu/PlusU1/BoundPlaneDim.lean](#) on GitHub.

def ExistsPlane[\[source\]](#)

A proposition which is true if for a given n a plane of charges of dimension n exists in which each point is a solution.

```
def ExistsPlane (n : ℕ) : Prop
```

2.16 HepLean.BeyondTheStandardModel.TwoHDM.Basic

[HepLean/BeyondTheStandardModel/TwoHDM/Basic.lean](#) on GitHub.

def potential[\[source\]](#)

The potential of the two Higgs doublet model.

```
def potential (Φ1 Φ2 : HiggsField)
  (m11Sq m22Sq lam1 lam2 lam3 lam4 : ℝ) (m12Sq lam5 lam6 lam7 : ℂ) (x : SpaceTime) : ℝ
```

2.17 HepLean.FeynmanDiagrams.Basic

[HepLean/FeynmanDiagrams/Basic.lean](#) on GitHub.

structure PreFeynmanRule[\[source\]](#)

A PreFeynmanRule is a set of rules specifying the allowed half-edges, edges and vertices in a diagram. (It does not specify how to turn the diagram into an algebraic expression.)

```
structure PreFeynmanRule where
  HalfEdgeLabel : Type
  EdgeLabel : Type
  VertexLabel : Type
  edgeLabelMap : EdgeLabel → CategoryTheory.Over HalfEdgeLabel
  vertexLabelMap : VertexLabel → CategoryTheory.Over HalfEdgeLabel
```

def toVertex[\[source\]](#)

The functor from $\text{Over } (P.\text{HalfEdgeLabel} \times P.\text{EdgeLabel} \times P.\text{VertexLabel})$ to $\text{Over } (P.\text{VertexLabel})$ induced by projections on products.

```
@[simps!]
def toVertex {E V : Type} : Over (P.HalfEdgeLabel × E × V) → Over V
```

def toEdge[\[source\]](#)

The functor from $\text{Over } (P.\text{HalfEdgeLabel} \times P.\text{EdgeLabel} \times P.\text{VertexLabel})$ to $\text{Over } (P.\text{EdgeLabel})$ induced by projections on products.

```
@[simps!]
def toEdge {E V : Type} : Over (P.HalfEdgeLabel × E × V) → Over E
```

def toHalfEdge[\[source\]](#)

The functor from $\text{Over } (P.\text{HalfEdgeLabel} \times P.\text{EdgeLabel} \times P.\text{VertexLabel})$ to $\text{Over } (P.\text{HalfEdgeLabel})$ induced by projections on products.

```
@[simps!]
def toHalfEdge {E V : Type} : Over (P.HalfEdgeLabel × E × V) → Over P.HalfEdgeLabel
```

def preimageType'

[\[source\]](#)

The functor from $\text{Over } P.\text{VertexLabel}$ to Type induced by pull-back along insertion of $v : P.\text{VertexLabel}$.

```
@[simps!]
def preimageType' {V : Type} (v : V) : Over V → Type where
```

def preimageVertex

[\[source\]](#)

The functor from $\text{Over } (P.\text{HalfEdgeLabel} \times P.\text{EdgeLabel} \times P.\text{VertexLabel})$ to $\text{Over } P.\text{HalfEdgeLabel}$ induced by pull-back along insertion of $v : P.\text{VertexLabel}$.

```
def preimageVertex {E V : Type} (v : V) :
  Over (P.HalfEdgeLabel × E × V) → Over P.HalfEdgeLabel where
```

def preimageEdge

[\[source\]](#)

The functor from $\text{Over } (P.\text{HalfEdgeLabel} \times P.\text{EdgeLabel} \times P.\text{VertexLabel})$ to $\text{Over } P.\text{HalfEdgeLabel}$ induced by pull-back along insertion of $v : P.\text{EdgeLabel}$.

```
def preimageEdge {E V : Type} (v : E) :
  Over (P.HalfEdgeLabel × E × V) → Over P.HalfEdgeLabel where
```

class IsFinitePreFeynmanRule

[\[source\]](#)

A set of conditions on PreFeynmanRule for it to be considered finite.

```
class IsFinitePreFeynmanRule (P : PreFeynmanRule) where
  edgeLabelDecidable : DecidableEq P.EdgeLabel
  vertexLabelDecidable : DecidableEq P.VertexLabel
  halfEdgeLabelDecidable : DecidableEq P.HalfEdgeLabel
  vertexMapFintype : ∀ v : P.VertexLabel, Fintype (P.vertexLabelMap v).left
  vertexMapDecidable : ∀ v : P.VertexLabel, DecidableEq (P.vertexLabelMap v).left
  edgeMapFintype : ∀ v : P.EdgeLabel, Fintype (P.edgeLabelMap v).left
  edgeMapDecidable : ∀ v : P.EdgeLabel, DecidableEq (P.edgeLabelMap v).left
```

instance preFeynmanRuleDecEqE

[\[source\]](#)

```
instance preFeynmanRuleDecEqE (P : PreFeynmanRule) [IsFinitePreFeynmanRule P] :
  DecidableEq P.EdgeLabel
```

instance preFeynmanRuleDecEqV

[\[source\]](#)

```
instance preFeynmanRuleDecEqV (P : PreFeynmanRule) [IsFinitePreFeynmanRule P] :
  DecidableEq P.VertexLabel
```

instance preFeynmanRuleDecEq $\langle \mathcal{E} \rangle$ [\[source\]](#)

```
instance preFeynmanRuleDecEq  $\langle \mathcal{E} \rangle$  (P : PreFeynmanRule) [IsFinitePreFeynmanRule P] :
  DecidableEq P.HalfEdgeLabel
```

instance Fintype (P.vertexLabelMap v).left [\[source\]](#)

```
instance [IsFinitePreFeynmanRule P] (v : P.VertexLabel) : Fintype (P.vertexLabelMap v).left
```

instance DecidableEq (P.vertexLabelMap v).left [\[source\]](#)

```
instance [IsFinitePreFeynmanRule P] (v : P.VertexLabel) : DecidableEq (P.vertexLabelMap v).left
```

instance Fintype (P.edgeLabelMap v).left [\[source\]](#)

```
instance [IsFinitePreFeynmanRule P] (v : P.EdgeLabel) : Fintype (P.edgeLabelMap v).left
```

instance DecidableEq (P.edgeLabelMap v).left [\[source\]](#)

```
instance [IsFinitePreFeynmanRule P] (v : P.EdgeLabel) : DecidableEq (P.edgeLabelMap v).left
```

instance preimageVertexDecidablePred [\[source\]](#)

```
instance preimageVertexDecidablePred { $\mathcal{E}$   $\mathcal{V}$  : Type} [DecidableEq  $\mathcal{V}$ ] (v :  $\mathcal{V}$ )
  (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ )) :
  DecidablePred fun x => x  $\in$  (P.toVertex.obj F).hom  $^{-1}$  {v}
```

instance preimageEdgeDecidablePred [\[source\]](#)

```
instance preimageEdgeDecidablePred { $\mathcal{E}$   $\mathcal{V}$  : Type} [DecidableEq  $\mathcal{E}$ ] (v :  $\mathcal{E}$ )
  (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ )) :
  DecidablePred fun x => x  $\in$  (P.toEdge.obj F).hom  $^{-1}$  {v}
```

instance preimageVertexDecidable [\[source\]](#)

```
instance preimageVertexDecidable { $\mathcal{E}$   $\mathcal{V}$  : Type} (v :  $\mathcal{V}$ )
  (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ )) [DecidableEq F.left] :
  DecidableEq ((P.preimageVertex v).obj F).left
```

instance preimageEdgeDecidable [\[source\]](#)

```
instance preimageEdgeDecidable { $\mathcal{E}$   $\mathcal{V}$  : Type} (v :  $\mathcal{E}$ )
  (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ )) [DecidableEq F.left] :
  DecidableEq ((P.preimageEdge v).obj F).left
```

instance preimageVertexFintype [\[source\]](#)

```
instance preimageVertexFintype { $\mathcal{E}$   $\mathcal{V}$  : Type} [DecidableEq  $\mathcal{V}$ ]
  (v :  $\mathcal{V}$ ) (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ )) [h : Fintype F.left] :
  Fintype ((P.preimageVertex v).obj F).left
```

instance preimageEdgeFintype[\[source\]](#)

```
instance preimageEdgeFintype {E V : Type} [DecidableEq E]
  (v : E) (F : Over (P.HalfEdgeLabel × E × V)) [h : Fintype F.left] :
  Fintype ((P.preimageEdge v).obj F).left
```

instance preimageVertexMapFintype[\[source\]](#)

```
instance preimageVertexMapFintype [IsFinitePreFeynmanRule P] {E V : Type}
  [DecidableEq V] (v : V) (f : V → P.VertexLabel) (F : Over (P.HalfEdgeLabel × E × V))
  [Fintype F.left] :
  Fintype ((P.vertexLabelMap (f v)).left → ((P.preimageVertex v).obj F).left)
```

instance preimageEdgeMapFintype[\[source\]](#)

```
instance preimageEdgeMapFintype [IsFinitePreFeynmanRule P] {E V : Type}
  [DecidableEq E] (v : E) (f : E → P.EdgeLabel) (F : Over (P.HalfEdgeLabel × E × V))
  [Fintype F.left] :
  Fintype ((P.edgeLabelMap (f v)).left → ((P.preimageEdge v).obj F).left)
```

def External[\[source\]](#)

A vertex is external if it has a single half-edge associated to it.

```
def External {P : PreFeynmanRule} (v : P.VertexLabel) : Prop
```

lemma external_iff_exists_bijection[\[source\]](#)

```
lemma external_iff_exists_bijection {P : PreFeynmanRule} (v : P.VertexLabel) :
  External v ↔ ∃ (κ : (P.vertexLabelMap v).left → Fin 1), Function.Bijective κ
```

instance externalDecidable[\[source\]](#)

```
instance externalDecidable [IsFinitePreFeynmanRule P] (v : P.VertexLabel) :
  Decidable (External v)
```

def DiagramVertexProp[\[source\]](#)

The proposition on vertices which must be satisfied by an object $F : \text{Over } (P.HalfEdgeLabel \times P.EdgeLabel \times P.VertexLabel)$ for it to be a Feynman diagram. This condition corresponds to the vertices of F having the correct half-edges associated with them.

```
def DiagramVertexProp {E V : Type} (F : Over (P.HalfEdgeLabel × E × V))
  (f : V → P.VertexLabel)
```

def DiagramEdgeProp[\[source\]](#)

The proposition on edges which must be satisfied by an object $F : \text{Over } (P.HalfEdgeLabel \times P.EdgeLabel \times P.VertexLabel)$ for it to be a Feynman diagram. This condition corresponds to the edges of F having the correct half-edges associated with them.

```
def DiagramEdgeProp {E V : Type} (F : Over (P.HalfEdgeLabel × E × V))
  (f : E → P.EdgeLabel)
```


lemma diagramVertexProp_iff[\[source\]](#)

```

lemma diagramVertexProp_iff { $\mathcal{E}$   $\mathcal{V}$  : Type} (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ ))
  (f :  $\mathcal{V} \longrightarrow$  P.VertexLabel) : P.DiagramVertexProp F f  $\leftrightarrow$ 
   $\forall$  v,  $\exists$  ( $\kappa$  : (P.vertexLabelMap (f v)).left  $\rightarrow$  ((P.preimageVertex v).obj F).left),
  Function.Bijective  $\kappa$ 
   $\wedge$  ((P.preimageVertex v).obj F).hom  $\circ$   $\kappa$  = (P.vertexLabelMap (f v)).hom

```

lemma diagramEdgeProp_iff[\[source\]](#)

```

lemma diagramEdgeProp_iff { $\mathcal{E}$   $\mathcal{V}$  : Type} (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ ))
  (f :  $\mathcal{E} \longrightarrow$  P.EdgeLabel) : P.DiagramEdgeProp F f  $\leftrightarrow$ 
   $\forall$  v,  $\exists$  ( $\kappa$  : (P.edgeLabelMap (f v)).left  $\rightarrow$  ((P.preimageEdge v).obj F).left),
  Function.Bijective  $\kappa$ 
   $\wedge$  ((P.preimageEdge v).obj F).hom  $\circ$   $\kappa$  = (P.edgeLabelMap (f v)).hom

```

instance diagramVertexPropDecidable[\[source\]](#)

```

instance diagramVertexPropDecidable
  [IsFinitePreFeynmanRule P] { $\mathcal{E}$   $\mathcal{V}$  : Type} [Fintype  $\mathcal{V}$ ] [DecidableEq  $\mathcal{V}$ ]
  (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ )) [DecidableEq F.left] [Fintype F.left]
  (f :  $\mathcal{V} \longrightarrow$  P.VertexLabel) : Decidable (P.DiagramVertexProp F f)

```

instance diagramEdgePropDecidable[\[source\]](#)

```

instance diagramEdgePropDecidable
  [IsFinitePreFeynmanRule P] { $\mathcal{E}$   $\mathcal{V}$  : Type} [Fintype  $\mathcal{E}$ ] [DecidableEq  $\mathcal{E}$ ]
  (F : Over (P.HalfEdgeLabel  $\times$   $\mathcal{E}$   $\times$   $\mathcal{V}$ )) [DecidableEq F.left] [Fintype F.left]
  (f :  $\mathcal{E} \longrightarrow$  P.EdgeLabel) : Decidable (P.DiagramEdgeProp F f)

```

structure FeynmanDiagram[\[source\]](#)

The type of Feynman diagrams given a PreFeynmanRule.

```

structure FeynmanDiagram (P : PreFeynmanRule) where
   $\mathcal{EO}$  : Over P.EdgeLabel
   $\mathcal{VO}$  : Over P.VertexLabel
   $\langle \mathcal{E} \text{To} \mathcal{E} \mathcal{V} \rangle$  : Over (P.HalfEdgeLabel  $\times$   $\mathcal{EO}$ .left  $\times$   $\mathcal{VO}$ .left)
   $\mathcal{E} \text{Cond}$  : P.DiagramEdgeProp  $\langle \mathcal{E} \text{To} \mathcal{E} \mathcal{V} \rangle$   $\mathcal{EO}$ .hom
   $\mathcal{V} \text{Cond}$  : P.DiagramVertexProp  $\langle \mathcal{E} \text{To} \mathcal{E} \mathcal{V} \rangle$   $\mathcal{VO}$ .hom

```

def FeynmanDiagram. $\mathcal{E}$ [\[source\]](#)

The type of edges.

```

def  $\mathcal{E}$  : Type

```

def FeynmanDiagram. $\mathcal{V}$ [\[source\]](#)

The type of vertices.

```

def  $\mathcal{V}$  : Type

```

def FeynmanDiagram. $\langle \mathcal{E}$ [\[source\]](#)

The type of half-edges.

def $\langle \mathcal{E}$: Type

def FeynmanDiagram. $\langle \mathcal{EO}$ [\[source\]](#)

The object in $\text{Over } P.\text{HalfEdgeLabel}$ generated by a Feynman diagram.

def $\langle \mathcal{EO}$: Over P.HalfEdgeLabel

def $\langle \mathcal{EToE}$ [\[source\]](#)

The map $F.\langle \mathcal{E} \rightarrow F.\mathcal{E}$ as an object in $\text{Over } F.\mathcal{E}$.

def $\langle \mathcal{EToE}$: Over F. $\mathcal{E}$

def $\langle \mathcal{EToV}$ [\[source\]](#)

The map $F.\langle \mathcal{E} \rightarrow F.\mathcal{V}$ as an object in $\text{Over } F.\mathcal{V}$.

def $\langle \mathcal{EToV}$: Over F. $\mathcal{V}$

def FeynmanDiagram.Cond [\[source\]](#)

The condition which must be satisfied by maps to form a Feynman diagram.

def Cond { $\mathcal{E} \mathcal{V} \langle \mathcal{E}$: Type} ($\mathcal{EO}$: $\mathcal{E} \rightarrow P.\text{EdgeLabel}$) ($\mathcal{VO}$: $\mathcal{V} \rightarrow P.\text{VertexLabel}$)
($\langle \mathcal{EToEV}$: $\langle \mathcal{E} \rightarrow P.\text{HalfEdgeLabel} \times \mathcal{E} \times \mathcal{V}$) : Prop

lemma cond_self [\[source\]](#)

lemma cond_self : Cond F. $\mathcal{EO}.\text{hom}$ F. $\mathcal{VO}.\text{hom}$ F. $\langle \mathcal{EToEV}.\text{hom}$

instance CondDecidable [\[source\]](#)

Cond is decidable.

instance CondDecidable [IsFinitePreFeynmanRule P] { $\mathcal{E} \mathcal{V} \langle \mathcal{E}$: Type} ($\mathcal{EO}$: $\mathcal{E} \rightarrow P.\text{EdgeLabel}$)
($\mathcal{VO}$: $\mathcal{V} \rightarrow P.\text{VertexLabel}$)
($\langle \mathcal{EToEV}$: $\langle \mathcal{E} \rightarrow P.\text{HalfEdgeLabel} \times \mathcal{E} \times \mathcal{V}$)
[Fintype $\mathcal{V}$] [DecidableEq $\mathcal{V}$] [Fintype $\mathcal{E}$] [DecidableEq $\mathcal{E}$] [h : Fintype $\langle \mathcal{E}$] [DecidableEq $\langle \mathcal{E}$]
:
Decidable (Cond $\mathcal{EO} \mathcal{VO} \langle \mathcal{EToEV}$)

def FeynmanDiagram.mk' [\[source\]](#)

Making a Feynman diagram from maps of edges, vertices and half-edges.

def mk' { $\mathcal{E} \mathcal{V} \langle \mathcal{E}$: Type} ($\mathcal{EO}$: $\mathcal{E} \rightarrow P.\text{EdgeLabel}$) ($\mathcal{VO}$: $\mathcal{V} \rightarrow P.\text{VertexLabel}$)
($\langle \mathcal{EToEV}$: $\langle \mathcal{E} \rightarrow P.\text{HalfEdgeLabel} \times \mathcal{E} \times \mathcal{V}$)
(C : Cond $\mathcal{EO} \mathcal{VO} \langle \mathcal{EToEV}$): FeynmanDiagram P **where**

lemma FeynmanDiagram.mk'_self[\[source\]](#)

```
lemma mk'_self : mk' F.ℰO.hom F.ℳO.hom F.⟨ℰToℰℳ.hom F.cond_self = F
```

class IsFiniteDiagram[\[source\]](#)

A set of conditions on a Feynman diagram for it to be considered finite.

```
class IsFiniteDiagram (F : FeynmanDiagram P) where
```

```
  ℰFintype : Fintype F.ℰ
  ℰDecidableEq : DecidableEq F.ℰ
  ℳFintype : Fintype F.ℳ
  ℳDecidableEq : DecidableEq F.ℳ
  ⟨ℰFintype : Fintype F.⟨ℰ
  ⟨ℰDecidableEq : DecidableEq F.⟨ℰ
```

instance IsFiniteDiagram (mk' ℰO ℳO ⟨ℰToℰℳ C)[\[source\]](#)

```
instance {ℰ ℳ ⟨ℰ : Type} [h1 : Fintype ℰ] [h2 : DecidableEq ℰ]
  [h3 : Fintype ℳ] [h4 : DecidableEq ℳ] [h5 : Fintype ⟨ℰ] [h6 : DecidableEq ⟨ℰ]
  (ℰO : ℰ → P.EdgeLabel) (ℳO : ℳ → P.VertexLabel)
  (⟨ℰToℰℳ : ⟨ℰ → P.HalfEdgeLabel × ℰ × ℳ) (C : Cond ℰO ℳO ⟨ℰToℰℳ) :
  IsFiniteDiagram (mk' ℰO ℳO ⟨ℰToℰℳ C)
```

instance Fintype F.ℰ[\[source\]](#)

```
instance {F : FeynmanDiagram P} [IsFiniteDiagram F] : Fintype F.ℰ
```

instance DecidableEq F.ℰ[\[source\]](#)

```
instance {F : FeynmanDiagram P} [IsFiniteDiagram F] : DecidableEq F.ℰ
```

instance Fintype F.ℳ[\[source\]](#)

```
instance {F : FeynmanDiagram P} [IsFiniteDiagram F] : Fintype F.ℳ
```

instance DecidableEq F.ℳ[\[source\]](#)

```
instance {F : FeynmanDiagram P} [IsFiniteDiagram F] : DecidableEq F.ℳ
```

instance Fintype F.⟨ℰ[\[source\]](#)

```
instance {F : FeynmanDiagram P} [IsFiniteDiagram F] : Fintype F.⟨ℰ
```

instance DecidableEq F.⟨ℰ[\[source\]](#)

```
instance {F : FeynmanDiagram P} [IsFiniteDiagram F] : DecidableEq F.⟨ℰ
```

instance Decidable (F.⟨ℰToℰℳ.hom i = j)[\[source\]](#)

```
instance {F : FeynmanDiagram P} [IsFiniteDiagram F] (i : F.⟨ℰ) (j : F.ℰ) :
  Decidable (F.⟨ℰToℰℳ.hom i = j)
```

instance fintypeProdHalfEdgeLabel $\mathcal{E}\mathcal{V}$ [\[source\]](#)

```
instance fintypeProdHalfEdgeLabel $\mathcal{E}\mathcal{V}$  {F : FeynmanDiagram P} [IsFinitePreFeynmanRule P]
  [IsFiniteDiagram F] : DecidableEq (P.HalfEdgeLabel × F. $\mathcal{E}$  × F. $\mathcal{V}$ )
```

def edgeVertexMap[\[source\]](#)

Given two maps $F.\mathcal{E} \rightarrow G.\mathcal{E}$ and $F.\mathcal{V} \rightarrow G.\mathcal{V}$ the corresponding map $P.HalfEdgeLabel \times F.\mathcal{E} \times F.\mathcal{V} \rightarrow P.HalfEdgeLabel \times G.\mathcal{E} \times G.\mathcal{V}$.

```
@[simps!]
```

```
def edgeVertexMap {F G : FeynmanDiagram P} ( $\mathcal{E}$  : F. $\mathcal{E} \rightarrow G.\mathcal{E}$ ) ( $\mathcal{V}$  : F. $\mathcal{V} \rightarrow G.\mathcal{V}$ ) :
  P.HalfEdgeLabel × F. $\mathcal{E}$  × F. $\mathcal{V} \rightarrow$  P.HalfEdgeLabel × G. $\mathcal{E}$  × G. $\mathcal{V}$ 
```

def edgeVertexEquiv[\[source\]](#)

Given equivalences $F.\mathcal{E} \approx G.\mathcal{E}$ and $F.\mathcal{V} \approx G.\mathcal{V}$, the induced equivalence between $P.HalfEdgeLabel \times F.\mathcal{E} \times F.\mathcal{V}$ and $P.HalfEdgeLabel \times G.\mathcal{E} \times G.\mathcal{V}$

```
def edgeVertexEquiv {F G : FeynmanDiagram P} ( $\mathcal{E}$  : F. $\mathcal{E} \approx G.\mathcal{E}$ ) ( $\mathcal{V}$  : F. $\mathcal{V} \approx G.\mathcal{V}$ ) :
  P.HalfEdgeLabel × F. $\mathcal{E}$  × F. $\mathcal{V} \approx$  P.HalfEdgeLabel × G. $\mathcal{E}$  × G. $\mathcal{V}$  where
```

def edgeVertexFunc[\[source\]](#)

The functor of over-categories generated by edgeVertexMap.

```
@[simps!]
```

```
def edgeVertexFunc {F G : FeynmanDiagram P} ( $\mathcal{E}$  : F. $\mathcal{E} \rightarrow G.\mathcal{E}$ ) ( $\mathcal{V}$  : F. $\mathcal{V} \rightarrow G.\mathcal{V}$ ) :
  Over (P.HalfEdgeLabel × F. $\mathcal{E}$  × F. $\mathcal{V}$ )  $\rightsquigarrow$  Over (P.HalfEdgeLabel × G. $\mathcal{E}$  × G. $\mathcal{V}$ )
```

structure Hom[\[source\]](#)

A morphism of Feynman diagrams.

```
structure Hom (F G : FeynmanDiagram P) where
   $\mathcal{EO}$  : F. $\mathcal{EO} \rightarrow G.\mathcal{EO}$ 
   $\mathcal{VO}$  : F. $\mathcal{VO} \rightarrow G.\mathcal{VO}$ 
   $\langle \mathcal{E} \text{To} \mathcal{E} \mathcal{V} \rangle$  : (edgeVertexFunc  $\mathcal{EO}$ .left  $\mathcal{VO}$ .left).obj F. $\langle \mathcal{E} \text{To} \mathcal{E} \mathcal{V} \rangle \rightarrow G.\langle \mathcal{E} \text{To} \mathcal{E} \mathcal{V} \rangle$ 
```

def Hom. $\mathcal{E}$ [\[source\]](#)

The map $F.\mathcal{E} \rightarrow G.\mathcal{E}$ induced by a homomorphism of Feynman diagrams.

```
@[simp]
```

```
def  $\mathcal{E}$  : F. $\mathcal{E} \rightarrow G.\mathcal{E}$ 
```

def Hom. $\mathcal{V}$ [\[source\]](#)

The map $F.\mathcal{V} \rightarrow G.\mathcal{V}$ induced by a homomorphism of Feynman diagrams.

```
@[simp]
```

```
def  $\mathcal{V}$  : F. $\mathcal{V} \rightarrow G.\mathcal{V}$ 
```

def Hom.⟨ $\mathcal{E}$ [\[source\]](#)

The map $F.\langle \mathcal{E} \rightarrow G.\langle \mathcal{E}$ induced by a homomorphism of Feynman diagrams.

```
@[simp]
def ⟨ $\mathcal{E}$  : F.⟨ $\mathcal{E} \rightarrow G.\langle \mathcal{E}$ 
```

def Hom.⟨ $\mathcal{EO}$ [\[source\]](#)

The morphism $F.\langle \mathcal{EO} \rightarrow G.\langle \mathcal{EO}$ induced by a homomorphism of Feynman diagrams.

```
@[simp]
def ⟨ $\mathcal{EO}$  : F.⟨ $\mathcal{EO} \rightarrow G.\langle \mathcal{EO}$ 
```

def id [\[source\]](#)

The identity morphism for a Feynman diagram.

```
def id (F : FeynmanDiagram P) : Hom F F where
```

def comp [\[source\]](#)

The composition of two morphisms of Feynman diagrams.

```
@[sims!  $\mathcal{EO\_left} \mathcal{VO\_left} \langle \mathcal{E} \text{To} \mathcal{E} \mathcal{V\_left}$ ]
def comp {F G H : FeynmanDiagram P} (f : Hom F G) (g : Hom G H) : Hom F H where
```

lemma ext' [\[source\]](#)

```
lemma ext' {F G : FeynmanDiagram P} {f g : Hom F G} (h $\mathcal{E}$  : f. $\mathcal{EO}$  = g. $\mathcal{EO}$ )
(h $\mathcal{V}$  : f. $\mathcal{VO}$  = g. $\mathcal{VO}$ ) (h⟨ $\mathcal{E}$  : f.⟨ $\mathcal{E}$  = g.⟨ $\mathcal{E}$ ) : f = g
```

lemma ext [\[source\]](#)

```
lemma ext {F G : FeynmanDiagram P} {f g : Hom F G} (h $\mathcal{E}$  : f. $\mathcal{E}$  = g. $\mathcal{E}$ )
(h $\mathcal{V}$  : f. $\mathcal{V}$  = g. $\mathcal{V}$ ) (h⟨ $\mathcal{E}$  : f.⟨ $\mathcal{E}$  = g.⟨ $\mathcal{E}$ ) : f = g
```

def Hom.Cond [\[source\]](#)

The condition on maps of edges, vertices and half-edges for it to be lifted to a morphism of Feynman diagrams.

```
def Cond {F G : FeynmanDiagram P} ( $\mathcal{E}$  : F. $\mathcal{E} \rightarrow G.\mathcal{E}$ ) ( $\mathcal{V}$  : F. $\mathcal{V} \rightarrow G.\mathcal{V}$ ) (⟨ $\mathcal{E}$  : F.⟨ $\mathcal{E} \rightarrow G.\langle \mathcal{E}$ ) : Prop
```

lemma cond_satisfied [\[source\]](#)

```
lemma cond_satisfied {F G : FeynmanDiagram P} (f : Hom F G) :
Cond f. $\mathcal{E}$  f. $\mathcal{V}$  f.⟨ $\mathcal{E}$ 
```

lemma cond_symm[\[source\]](#)

```
lemma cond_symm {F G : FeynmanDiagram P} (E : F.E = G.E) (V : F.V = G.V) (⟨E : F.⟨E = G.⟨E)
  (h : Cond E V ⟨E) : Cond E.symm V.symm ⟨E.symm
```

instance Decidable (∀ x, G.EO.hom (E x) = F.EO.hom x)[\[source\]](#)

```
instance {F G : FeynmanDiagram P} [IsFiniteDiagram F] [IsFinitePreFeynmanRule P]
  (E : F.E → G.E) : Decidable (∀ x, G.EO.hom (E x) = F.EO.hom x)
```

instance Decidable (∀ x, G.VO.hom (V x) = F.VO.hom x)[\[source\]](#)

```
instance {F G : FeynmanDiagram P} [IsFiniteDiagram F] [IsFinitePreFeynmanRule P]
  (V : F.V → G.V) : Decidable (∀ x, G.VO.hom (V x) = F.VO.hom x)
```

instance Decidable (∀ x, G.⟨EToEV.hom (⟨E x) = edgeVertexMap E V (F.⟨E...[\[source\]](#)

```
instance {F G : FeynmanDiagram P} [IsFiniteDiagram F] [IsFiniteDiagram G] [
  IsFinitePreFeynmanRule P]
  (E : F.E → G.E) (V : F.V → G.V) (⟨E : F.⟨E → G.⟨E) :
  Decidable (∀ x, G.⟨EToEV.hom (⟨E x) = edgeVertexMap E V (F.⟨EToEV.hom x))
```

instance Decidable (Cond E V ⟨E)[\[source\]](#)

```
instance {F G : FeynmanDiagram P} [IsFiniteDiagram F] [IsFiniteDiagram G] [
  IsFinitePreFeynmanRule P]
  (E : F.E → G.E) (V : F.V → G.V) (⟨E : F.⟨E → G.⟨E) : Decidable (Cond E V ⟨E)
```

def Hom.mk'[\[source\]](#)

Making a Feynman diagram from maps of edges, vertices and half-edges.

```
@[simps! EO_left VO_left ⟨EToEV_left]
def mk' {F G : FeynmanDiagram P} (E : F.E → G.E) (V : F.V → G.V) (⟨E : F.⟨E → G.⟨E)
  (C : Cond E V ⟨E) : Hom F G where
```

lemma Hom.mk'_self[\[source\]](#)

```
lemma mk'_self {F G : FeynmanDiagram P} (f : Hom F G) :
  mk' f.E f.V f.⟨E f.cond_satisfied = f
```

instance Category (FeynmanDiagram P)[\[source\]](#)

Feynman diagrams form a category.

```
@[simps! id_EO_left id_VO_left id_⟨EToEV_left comp_EO_left comp_VO_left comp_⟨EToEV_left]
instance : Category (FeynmanDiagram P) where
```

def mkIso[\[source\]](#)

An isomorphism of Feynman diagrams from isomorphisms of edges, vertices and half-edges.

```
def mkIso {F G : FeynmanDiagram P} (E : F.E ≈ G.E) (V : F.V ≈ G.V) (⟨E : F.⟨E ≈ G.⟨E)
  (C : Hom.Cond E V ⟨E) : F ≅ G where
```

def funcEO

[\[source\]](#)

The functor from Feynman diagrams to category over edge labels.

```
def funcEO : FeynmanDiagram P ↗ Over P.EdgeLabel where
```

def funcVO

[\[source\]](#)

The functor from Feynman diagrams to category over vertex labels.

```
def funcVO : FeynmanDiagram P ↗ Over P.VertexLabel where
```

def func⟨EO

[\[source\]](#)

The functor from Feynman diagrams to category over half-edge labels.

```
def func⟨EO : FeynmanDiagram P ↗ Over P.HalfEdgeLabel where
```

def funcE

[\[source\]](#)

The functor from Feynman diagrams to Type landing on edges.

```
def funcE : FeynmanDiagram P ↗ Type where
```

def funcV

[\[source\]](#)

The functor from Feynman diagrams to Type landing on vertices.

```
def funcV : FeynmanDiagram P ↗ Type where
```

def func⟨E

[\[source\]](#)

The functor from Feynman diagrams to Type landing on half-edges.

```
def func⟨E : FeynmanDiagram P ↗ Type where
```

def SymmetryType

[\[source\]](#)

The type of isomorphisms of a Feynman diagram.

```
def SymmetryType : Type
```

def symmetryTypeEquiv

[\[source\]](#)

An equivalence between SymmetryType and permutation of edges, vertices and half-edges satisfying Hom.Cond.

```
def symmetryTypeEquiv :
  F.SymmetryType ≈ {S : Equiv.Perm F.E × Equiv.Perm F.V × Equiv.Perm F.⟨E //
    Hom.Cond S.1 S.2.1 S.2.2} where
```

instance Fintype F.SymmetryType[\[source\]](#)

```
instance [IsFinitePreFeynmanRule P] [IsFiniteDiagram F] : Fintype F.SymmetryType
```

def cardSymmetryFactor[\[source\]](#)

The symmetry factor can be defined as the cardinal of the symmetry type. In general this is not a finite number.

```
@[simp]
def cardSymmetryFactor : Cardinal
```

def symmetryFactor[\[source\]](#)

The symmetry factor of a Finite Feynman diagram, as a natural number.

```
@[simp]
def symmetryFactor [IsFinitePreFeynmanRule P] [IsFiniteDiagram F] : ℕ
```

lemma symmetryFactor_eq_cardSymmetryFactor[\[source\]](#)

```
@[simp]
lemma symmetryFactor_eq_cardSymmetryFactor [IsFinitePreFeynmanRule P] [IsFiniteDiagram F] :
  F.symmetryFactor = F.cardSymmetryFactor
```

def AdjRelation[\[source\]](#)

A relation on the vertices of Feynman diagrams. The proposition is true if the two vertices are not equal and are connected by a single edge.

```
@[simp]
def AdjRelation : F.ℳ → F.ℳ → Prop
```

instance DecidableRel F.AdjRelation[\[source\]](#)

```
instance [IsFiniteDiagram F] : DecidableRel F.AdjRelation
```

def toSimpleGraph[\[source\]](#)

From a Feynman diagram the simple graph showing those vertices which are connected.

```
def toSimpleGraph : SimpleGraph F.ℳ where
```

instance DecidableRel F.toSimpleGraph.Adj[\[source\]](#)

```
instance [IsFiniteDiagram F] : DecidableRel F.toSimpleGraph.Adj
```

instance Decidable (F.toSimpleGraph.Preconnected ∧ Nonempty F.ℳ)[\[source\]](#)

```
instance [IsFiniteDiagram F] :
  Decidable (F.toSimpleGraph.Preconnected ∧ Nonempty F.ℳ)
```


instance Decidable F.toSimpleGraph.Connected [\[source\]](#)

instance [IsFiniteDiagram F] : Decidable F.toSimpleGraph.Connected

def Connected [\[source\]](#)

A Feynman diagram is connected if its simple graph is connected.

def Connected : Prop

instance Decidable (Connected F) [\[source\]](#)

instance [IsFiniteDiagram F] : Decidable (Connected F)

2.18 HepLean.FeynmanDiagrams.Instances.ComplexScalar

[HepLean/FeynmanDiagrams/Instances/ComplexScalar.lean](#) on GitHub.

def complexScalarFeynmanRules [\[source\]](#)

The pre-Feynman rules for a complex scalar theory.

@[simps!]

def complexScalarFeynmanRules : PreFeynmanRule **where**

instance OfNat complexScalarFeynmanRules.EdgeLabel a [\[source\]](#)

instance (a : ℕ) : OfNat complexScalarFeynmanRules.EdgeLabel a **where**

instance OfNat complexScalarFeynmanRules.HalfEdgeLabel a [\[source\]](#)

instance (a : ℕ) : OfNat complexScalarFeynmanRules.HalfEdgeLabel a **where**

instance OfNat complexScalarFeynmanRules.VertexLabel a [\[source\]](#)

instance (a : ℕ) : OfNat complexScalarFeynmanRules.VertexLabel a **where**

instance IsFinitePreFeynmanRule complexScalarFeynmanRules [\[source\]](#)

instance : IsFinitePreFeynmanRule complexScalarFeynmanRules **where**

2.19 HepLean.FeynmanDiagrams.Instances.Phi4

[HepLean/FeynmanDiagrams/Instances/Phi4.lean](#) on GitHub.

def phi4PreFeynmanRules [\[source\]](#)

The pre-Feynman rules for Φ^4 theory.

@[simps!]

def phi4PreFeynmanRules : PreFeynmanRule **where**

instance OfNat phi4PreFeynmanRules.EdgeLabel a [\[source\]](#)

instance (a : $\mathbb{N}$) : OfNat phi4PreFeynmanRules.EdgeLabel a **where**

instance OfNat phi4PreFeynmanRules.HalfEdgeLabel a [\[source\]](#)

instance (a : $\mathbb{N}$) : OfNat phi4PreFeynmanRules.HalfEdgeLabel a **where**

instance OfNat phi4PreFeynmanRules.VertexLabel a [\[source\]](#)

instance (a : $\mathbb{N}$) : OfNat phi4PreFeynmanRules.VertexLabel a **where**

instance IsFinitePreFeynmanRule phi4PreFeynmanRules [\[source\]](#)

instance : IsFinitePreFeynmanRule phi4PreFeynmanRules **where**

abbrev figureEight [\[source\]](#)

The figure eight Feynman diagram.

abbrev figureEight : FeynmanDiagram phi4PreFeynmanRules

lemma figureEight_connected [\[source\]](#)

figureEight is connected. We can get this from #eval Connected figureEight.

lemma figureEight_connected : Connected figureEight

lemma figureEight_symmetryFactor [\[source\]](#)

The symmetry factor of figureEight is 8. We can get this from #eval symmetryFactor figureEight.

lemma figureEight_symmetryFactor : symmetryFactor figureEight = 8

2.20 HepLean.FeynmanDiagrams.Momentum

[HepLean/FeynmanDiagrams/Momentum.lean](#) on GitHub.

def HalfEdgeMomenta [\[source\]](#)

The type which associates to each half-edge a 1-dimensional vector space. Corresponding to that spanned by its momentum.

def HalfEdgeMomenta : Type

instance AddCommGroup F.HalfEdgeMomenta [\[source\]](#)

instance : AddCommGroup F.HalfEdgeMomenta

instance Module $\mathbb{R}$ F.HalfEdgeMomenta

[\[source\]](#)

instance : Module $\mathbb{R}$ F.HalfEdgeMomenta

def euclidInnerAux

[\[source\]](#)

An auxillary function used to define the Euclidean inner product on F.HalfEdgeMomenta.

def euclidInnerAux (x : F.HalfEdgeMomenta) : F.HalfEdgeMomenta $\rightarrow_l$ $\mathbb{R}$ **where**

lemma euclidInnerAux_symm

[\[source\]](#)

lemma euclidInnerAux_symm (x y : F.HalfEdgeMomenta) :
F.euclidInnerAux x y = F.euclidInnerAux y x

def euclidInner

[\[source\]](#)

The Euclidean inner product on F.HalfEdgeMomenta.

def euclidInner : F.HalfEdgeMomenta $\rightarrow_l$ $\mathbb{R}$ F.HalfEdgeMomenta $\rightarrow_l$ $\mathbb{R}$ **where**

def EdgeMomenta

[\[source\]](#)

The type which assocaites to each ege a 1-dimensional vector space. Corresponding to that spanned by its total outflowing momentum.

def EdgeMomenta : Type

instance AddCommGroup F.EdgeMomenta

[\[source\]](#)

instance : AddCommGroup F.EdgeMomenta

instance Module $\mathbb{R}$ F.EdgeMomenta

[\[source\]](#)

instance : Module $\mathbb{R}$ F.EdgeMomenta

def VertexMomenta

[\[source\]](#)

The type which assocaites to each ege a 1-dimensional vector space. Corresponding to that spanned by its total inflowing momentum.

def VertexMomenta : Type

instance AddCommGroup F.VertexMomenta

[\[source\]](#)

instance : AddCommGroup F.VertexMomenta

instance Module $\mathbb{R}$ F.VertexMomenta

[\[source\]](#)

instance : Module $\mathbb{R}$ F.VertexMomenta

def EdgeVertexMomentaMap[\[source\]](#)

The map from $\text{Fin } 2$ to Type landing on EdgeMomenta and VertexMomenta .

```
def EdgeVertexMomentaMap : Fin 2 → Type
```

instance AddCommGroup (EdgeVertexMomentaMap F i)[\[source\]](#)

```
instance (i : Fin 2) : AddCommGroup (EdgeVertexMomentaMap F i)
```

instance Module ℝ (EdgeVertexMomentaMap F i)[\[source\]](#)

```
instance (i : Fin 2) : Module ℝ (EdgeVertexMomentaMap F i)
```

def EdgeVertexMomenta[\[source\]](#)

The direct sum of EdgeMomenta and VertexMomenta .

```
def EdgeVertexMomenta : Type
```

instance AddCommGroup F.EdgeVertexMomenta[\[source\]](#)

```
instance : AddCommGroup F.EdgeVertexMomenta
```

instance Module ℝ F.EdgeVertexMomenta[\[source\]](#)

```
instance : Module ℝ F.EdgeVertexMomenta
```

def edgeToHalfEdgeMomenta[\[source\]](#)

The linear map from $F.\text{EdgeMomenta}$ to $F.\text{HalfEdgeMomenta}$ induced by the map $F.\langle \mathcal{E} \text{To} \mathcal{E} \rangle.\text{hom}$.

```
def edgeToHalfEdgeMomenta : F.EdgeMomenta →ℓ[ℝ] F.HalfEdgeMomenta where
```

def vertexToHalfEdgeMomenta[\[source\]](#)

The linear map from $F.\text{VertexMomenta}$ to $F.\text{HalfEdgeMomenta}$ induced by the map $F.\langle \mathcal{E} \text{To} \mathcal{V} \rangle.\text{hom}$.

```
def vertexToHalfEdgeMomenta : F.VertexMomenta →ℓ[ℝ] F.HalfEdgeMomenta where
```

def edgeVertexToHalfEdgeMomenta[\[source\]](#)

The linear map from $F.\text{EdgeVertexMomenta}$ to $F.\text{HalfEdgeMomenta}$ induced by $F.\text{edgeToHalfEdgeMomenta}$ and $F.\text{vertexToHalfEdgeMomenta}$.

```
def edgeVertexToHalfEdgeMomenta : F.EdgeVertexMomenta →ℓ[ℝ] F.HalfEdgeMomenta
```

def orthogHalfEdgeMomenta[\[source\]](#)

The submodule of $F.\text{HalfEdgeMomenta}$ corresponding to the range of $F.\text{edgeVertexToHalfEdgeMomenta}$.

```
def orthogHalfEdgeMomenta : Submodule ℝ F.HalfEdgeMomenta
```

def allowedHalfEdgeMomenta[\[source\]](#)

The submodule of $F.HalfEdgeMomenta$ corresponding to the allowed momenta.

```
def allowedHalfEdgeMomenta : Submodule ℝ F.HalfEdgeMomenta
```

def numberOfLoops[\[source\]](#)

The number of loops of a Feynman diagram. Defined as the dimension of the space of allowed Half-loop momenta.

```
noncomputable def numberOfLoops : ℕ
```

2.21 HepLean.FlavorPhysics.CKMMatrix.Basic

[HepLean/FlavorPhysics/CKMMatrix/Basic.lean](#) on GitHub.

def PhaseShiftRelation[\[source\]](#)

The equivalence relation between CKM matrices.

```
def PhaseShiftRelation (U V : unitaryGroup (Fin 3) ℂ) : Prop
```

2.22 HepLean.FlavorPhysics.CKMMatrix.PhaseFreedom

[HepLean/FlavorPhysics/CKMMatrix/PhaseFreedom.lean](#) on GitHub.

def FstRowThdColRealCond[\[source\]](#)

A proposition which is satisfied by a CKM matrix if its ud , us , cb and tb elements are positive and real, and there is no phase difference between the t th-row and the cross product of the conjugates of the u th and c th rows.

```
def FstRowThdColRealCond (U : CKMMatrix) : Prop
```

2.23 HepLean.SpaceTime.AsSelfAdjointMatrix

[HepLean/SpaceTime/AsSelfAdjointMatrix.lean](#) on GitHub.

def toMatrix[\[source\]](#)

A 2×2 -complex matrix formed from a space-time point.

```
@[simp]
def toMatrix (x : SpaceTime) : Matrix (Fin 2) (Fin 2) ℂ
```

lemma toMatrix_isSelfAdjoint[\[source\]](#)

The matrix $x.toMatrix$ for $x \in spaceTime$ is self adjoint.

```
lemma toMatrix_isSelfAdjoint (x : SpaceTime) : IsSelfAdjoint x.toMatrix
```

def toSelfAdjointMatrix'[\[source\]](#)*A self-adjoint matrix formed from a space-time point.*`@[simps!]``def toSelfAdjointMatrix' (x : SpaceTime) : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)`**def fromSelfAdjointMatrix'**[\[source\]](#)*A self-adjoint matrix formed from a space-time point.*`@[simp]``noncomputable def fromSelfAdjointMatrix' (x : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
 SpaceTime`**def toSelfAdjointMatrix**[\[source\]](#)*The linear equivalence between the vector-space `spaceTime` and self-adjoint  $2 \times 2$ -complex matrices.*`noncomputable def toSelfAdjointMatrix : SpaceTime $\approx_l$ [ℝ] selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)
 where`**lemma det\_eq\_ηLin**[\[source\]](#)`lemma det_eq_ηLin (x : SpaceTime) : det (toSelfAdjointMatrix x).1 = ηLin x x`**2.24 HepLean.SpaceTime.Basic**[HepLean/SpaceTime/Basic.lean](#) on GitHub.**def SpaceTime**[\[source\]](#)*The space-time*`def SpaceTime : Type`**instance AddCommMonoid SpaceTime**[\[source\]](#)*Give spacetime the structure of an additive commutative monoid.*`instance : AddCommMonoid SpaceTime`**instance Module ℝ SpaceTime**[\[source\]](#)*Give spacetime the structure of a module over the reals.*`instance : Module ℝ SpaceTime`**def space**[\[source\]](#)*The space part of spacetime.*`@[simp]``def space (x : SpaceTime) : EuclideanSpace ℝ (Fin 3)`

def asSmoothManifold [\[source\]](#)

The structure of a smooth manifold on spacetime.

```
def asSmoothManifold : ModelWithCorners ℝ SpaceTime SpaceTime
```

instance ChartedSpace SpaceTime SpaceTime [\[source\]](#)

```
instance : ChartedSpace SpaceTime SpaceTime
```

def stdBasis [\[source\]](#)

The standard basis for spacetime.

```
def stdBasis : Basis (Fin 4) ℝ SpaceTime
```

lemma stdBasis_apply [\[source\]](#)

```
lemma stdBasis_apply (μ v : Fin 4) : stdBasis μ v = if μ = v then 1 else 0
```

lemma stdBasis_not_eq [\[source\]](#)

```
lemma stdBasis_not_eq {μ v : Fin 4} (h : μ ≠ v) : stdBasis μ v = 0
```

lemma stdBasis_0 [\[source\]](#)

```
lemma stdBasis_0 : stdBasis 0 = ![1, 0, 0, 0]
```

lemma stdBasis_1 [\[source\]](#)

```
lemma stdBasis_1 : stdBasis 1 = ![0, 1, 0, 0]
```

lemma stdBasis_2 [\[source\]](#)

```
lemma stdBasis_2 : stdBasis 2 = ![0, 0, 1, 0]
```

lemma stdBasis_3 [\[source\]](#)

```
lemma stdBasis_3 : stdBasis 3 = ![0, 0, 0, 1]
```

lemma stdBasis_mulVec [\[source\]](#)

```
lemma stdBasis_mulVec (μ v : Fin 4) (Λ : Matrix (Fin 4) (Fin 4) ℝ) :  
  (Λ *v stdBasis μ) v = Λ v μ
```

lemma explicit [\[source\]](#)

```
lemma explicit (x : SpaceTime) : x = ![x 0, x 1, x 2, x 3]
```

lemma add_apply[\[source\]](#)

@[simp]

lemma add_apply (x y : SpaceTime) (i : Fin 4) : (x + y) i = x i + y i**lemma smul_apply**[\[source\]](#)

@[simp]

lemma smul_apply (x : SpaceTime) (a : ℝ) (i : Fin 4) : (a • x) i = a * x i**2.25 HepLean.SpaceTime.LorentzAlgebra.Basic**[HepLean/SpaceTime/LorentzAlgebra/Basic.lean](#) on GitHub.**def lorentzAlgebra**[\[source\]](#)*The Lorentz algebra as a subalgebra of $\text{Matrix (Fin 4) (Fin 4) } \mathbb{R}$.***def** lorentzAlgebra : LieSubalgebra ℝ (Matrix (Fin 4) (Fin 4) ℝ)**lemma transpose_eta**[\[source\]](#)**lemma** transpose_eta (A : lorentzAlgebra) : $A.1^T * \eta = - \eta * A.1$ **lemma mem_of_transpose_eta_eq_eta_mul_self**[\[source\]](#)**lemma** mem_of_transpose_eta_eq_eta_mul_self {A : Matrix (Fin 4) (Fin 4) ℝ}
(h : $A^T * \eta = - \eta * A$) : A ∈ lorentzAlgebra**lemma mem_iff**[\[source\]](#)**lemma** mem_iff {A : Matrix (Fin 4) (Fin 4) ℝ} : A ∈ lorentzAlgebra ↔ $A^T * \eta = - \eta * A$ **lemma mem_iff'**[\[source\]](#)**lemma** mem_iff' (A : Matrix (Fin 4) (Fin 4) ℝ) : A ∈ lorentzAlgebra ↔ $A = - \eta * A^T * \eta$ **lemma diag_comp**[\[source\]](#)**lemma** diag_comp (Λ : lorentzAlgebra) (μ : Fin 4) : $\Lambda.1 \mu \mu = 0$ **lemma time_comps**[\[source\]](#)**lemma** time_comps (Λ : lorentzAlgebra) (i : Fin 3) : $\Lambda.1 i.succ 0 = \Lambda.1 0 i.succ$ **lemma space_comps**[\[source\]](#)**lemma** space_comps (Λ : lorentzAlgebra) (i j : Fin 3) :
 $\Lambda.1 i.succ j.succ = - \Lambda.1 j.succ i.succ$

instance spaceTimeAsLieRingModule[\[source\]](#)

```
@[simps!]
instance spaceTimeAsLieRingModule : LieRingModule lorentzAlgebra SpaceTime where
```

instance spaceTimeAsLieModule[\[source\]](#)

```
@[simps!]
instance spaceTimeAsLieModule : LieModule ℝ lorentzAlgebra SpaceTime where
```

2.26 HepLean.SpaceTime.LorentzAlgebra.Basis

[HepLean/SpaceTime/LorentzAlgebra/Basis.lean](#) on GitHub.

def σMat[\[source\]](#)

The matrices which form the basis of the Lorentz algebra.

```
@[simp]
def σMat (μ ν : Fin 4) : Matrix (Fin 4) (Fin 4) ℝ
```

lemma σMat_in_lorentzAlgebra[\[source\]](#)

```
lemma σMat_in_lorentzAlgebra (μ ν : Fin 4) : σMat μ ν ∈ lorentzAlgebra
```

def σ[\[source\]](#)

Elements of the Lorentz algebra which form a basis thereof.

```
@[simps!]
def σ (μ ν : Fin 4) : lorentzAlgebra
```

lemma σ_anti_symm[\[source\]](#)

```
lemma σ_anti_symm (μ ν : Fin 4) : σ μ ν = - σ ν μ
```

lemma σMat_mul[\[source\]](#)

```
lemma σMat_mul (α β γ δ a b : Fin 4) :
  (σMat α β * σMat γ δ) a b =
  η^[a]_[α] * (η_[δ]_[b] * η_[β]_[γ] - η_[γ]_[b] * η_[β]_[δ])
  - η^[a]_[β] * (η_[δ]_[b] * η_[α]_[γ] - η_[γ]_[b] * η_[α]_[δ])
```

lemma σ_comm[\[source\]](#)

```
lemma σ_comm (α β γ δ : Fin 4) :
  {σ α β, σ γ δ} =
  η_[α]_[δ] • σ γ β + η_[α]_[γ] • σ β δ + η_[β]_[δ] • σ α γ + η_[β]_[γ] • σ δ α
```

lemma eq_span_σ[\[source\]](#)

```

lemma eq_span_σ (Λ : lorentzAlgebra) :
  Λ = Λ.1 0 1 • σ 0 1 + Λ.1 0 2 • σ 0 2 + Λ.1 0 3 • σ 0 3 +
    Λ.1 1 2 • σ 1 2 + Λ.1 1 3 • σ 1 3 + Λ.1 2 3 • σ 2 3

```

def σCoordinateMap[\[source\]](#)

The coordinate map for the basis formed by the matrices σ.

```

@[simps!]
noncomputable def σCoordinateMap : lorentzAlgebra ≈ℓ[ℝ] Fin 6 →σ ℝ where

```

def σBasis[\[source\]](#)

The basis formed by the matrices σ.

```

@[simps! repr_apply_support_val repr_apply_toFun]
noncomputable def σBasis : Basis (Fin 6) ℝ lorentzAlgebra where

```

instance Module.Finite ℝ lorentzAlgebra[\[source\]](#)

```

instance : Module.Finite ℝ lorentzAlgebra

```

theorem finrank_eq_six[\[source\]](#)

The Lorentz algebra is 6-dimensional.

```

theorem finrank_eq_six : FiniteDimensional.finrank ℝ lorentzAlgebra = 6

```

2.27 HepLean.SpaceTime.LorentzGroup.Basic

[HepLean/SpaceTime/LorentzGroup/Basic.lean](#) on GitHub.

def PreservesηLin[\[source\]](#)

*We say a matrix Λ preserves ηLin if for all x and y, ηLin (Λ *_v x) (Λ *_v y) = ηLin x y.*

```

def PreservesηLin (Λ : Matrix (Fin 4) (Fin 4) ℝ) : Prop

```

lemma iff_norm[\[source\]](#)

```

lemma iff_norm : PreservesηLin Λ ↔
  ∀ (x : SpaceTime), ηLin (Λ *v x) (Λ *v x) = ηLin x x

```

lemma iff_det_selfAdjoint[\[source\]](#)

```

lemma iff_det_selfAdjoint : PreservesηLin Λ ↔
  ∀ (x : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)),
  det ((toSelfAdjointMatrix ◦ toLin stdBasis stdBasis Λ ◦ toSelfAdjointMatrix.symm) x).1
  = det x.1

```

lemma iff_on_right[\[source\]](#)

```
lemma iff_on_right : PreservesηLin Λ ↔
  ∀ (x y : SpaceTime), ηLin x ((η * ΛT * η * Λ) *v y) = ηLin x y
```

lemma iff_matrix[\[source\]](#)

```
lemma iff_matrix : PreservesηLin Λ ↔ η * ΛT * η * Λ = 1
```

lemma iff_matrix'[\[source\]](#)

```
lemma iff_matrix' : PreservesηLin Λ ↔ Λ * (η * ΛT * η) = 1
```

lemma iff_transpose[\[source\]](#)

```
lemma iff_transpose : PreservesηLin Λ ↔ PreservesηLin ΛT
```

def liftGL[\[source\]](#)

The lift of a matrix which preserves ηLin to an invertible matrix.

```
def liftGL {Λ : Matrix (Fin 4) (Fin 4) ℝ} (h : PreservesηLin Λ) : GL (Fin 4) ℝ
```

lemma mul[\[source\]](#)

```
lemma mul {Λ Λ' : Matrix (Fin 4) (Fin 4) ℝ} (h : PreservesηLin Λ) (h' : PreservesηLin Λ') :
  PreservesηLin (Λ * Λ')
```

lemma one[\[source\]](#)

```
lemma one : PreservesηLin 1
```

lemma η[\[source\]](#)

```
lemma η : PreservesηLin η
```

def LorentzGroup[\[source\]](#)

The Lorentz group is the subset of matrices which preserve ηLin.

```
def LorentzGroup : Type
```

instance lorentzGroupIsGroup[\[source\]](#)

```
@[simps mul_coe one_coe inv div]
instance lorentzGroupIsGroup : Group LorentzGroup where
```

instance TopologicalSpace LorentzGroup[\[source\]](#)

lorentzGroup has the subtype topology.

```
instance : TopologicalSpace LorentzGroup
```

```
lemma coe_inv
```

[\[source\]](#)

```
lemma coe_inv (A : LorentzGroup) : (A-1).1 = A.1-1
```

```
def transpose
```

[\[source\]](#)

The transpose of an matrix in the Lorentz group is an element of the Lorentz group.

```
def transpose (Λ : LorentzGroup) : LorentzGroup
```

```
def toGL
```

[\[source\]](#)

The homomorphism of the Lorentz group into GL (Fin 4) ℝ.

```
def toGL : LorentzGroup →* GL (Fin 4) ℝ where
```

```
lemma toGL_injective
```

[\[source\]](#)

```
lemma toGL_injective : Function.Injective toGL
```

```
def toProd
```

[\[source\]](#)

The homomorphism from the Lorentz Group into the monoid of matrices times the opposite of the monoid of matrices.

```
@[simps!]
```

```
def toProd : LorentzGroup →* (Matrix (Fin 4) (Fin 4) ℝ) × (Matrix (Fin 4) (Fin 4) ℝ)mop
```

```
lemma toProd_eq_transpose_η
```

[\[source\]](#)

```
lemma toProd_eq_transpose_η : toProd A = (A.1, {η * A.1T * η})
```

```
lemma toProd_injective
```

[\[source\]](#)

```
lemma toProd_injective : Function.Injective toProd
```

```
lemma toProd_continuous
```

[\[source\]](#)

```
lemma toProd_continuous : Continuous toProd
```

```
lemma toProd_embedding
```

[\[source\]](#)

The embedding from the Lorentz Group into the monoid of matrices times the opposite of the monoid of matrices.

```
lemma toProd_embedding : Embedding toProd where
```

lemma toGL_embedding

[\[source\]](#)

The embedding from the Lorentz Group into GL (Fin 4) $\mathbb{R}$.

```
lemma toGL_embedding : Embedding toGL.toFun where
```

instance TopologicalGroup LorentzGroup

[\[source\]](#)

```
instance : TopologicalGroup LorentzGroup
```

2.28 HepLean.SpaceTime.LorentzGroup.Boosts

[HepLean/SpaceTime/LorentzGroup/Boosts.lean](#) on GitHub.

def genBoostAux₁

[\[source\]](#)

An auxillary linear map used in the definition of a generalised boost.

```
def genBoostAux1 (u v : FourVelocity) : SpaceTime →l  $\mathbb{R}$  SpaceTime where
```

def genBoostAux₂

[\[source\]](#)

An auxillary linear map used in the definition of a generalised boost.

```
def genBoostAux2 (u v : FourVelocity) : SpaceTime →l  $\mathbb{R}$  SpaceTime where
```

def genBoost

[\[source\]](#)

An generalised boost. This is a Lorentz transformation which takes the four velocity u to v .

```
def genBoost (u v : FourVelocity) : SpaceTime →l  $\mathbb{R}$  SpaceTime
```

lemma self

[\[source\]](#)

This lemma states that for a given four-velocity u , the general boost transformation $\text{genBoost } u$ is equal to the identity linear map LinearMap.id .

```
lemma self (u : FourVelocity) : genBoost u u = LinearMap.id
```

def toMatrix

[\[source\]](#)

A generalised boost as a matrix.

```
def toMatrix (u v : FourVelocity) : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ 
```

lemma toMatrix_mulVec

[\[source\]](#)

```
lemma toMatrix_mulVec (u v : FourVelocity) (x : SpaceTime) :  
  (toMatrix u v).mulVec x = genBoost u v x
```

lemma toMatrix_apply[\[source\]](#)

```
lemma toMatrix_apply (u v : FourVelocity) (i j : Fin 4) :
  (toMatrix u v) i j =
    η i i * (ηLin (stdBasis i) (stdBasis j) + 2 * ηLin (stdBasis j) u * ηLin (stdBasis i) v -
      ηLin (stdBasis i) (u + v) * ηLin (stdBasis j) (u + v) / (1 + ηLin u v))
```

lemma toMatrix_continuous[\[source\]](#)

```
lemma toMatrix_continuous (u : FourVelocity) : Continuous (toMatrix u)
```

lemma toMatrix_PreservesηLin[\[source\]](#)

```
lemma toMatrix_PreservesηLin (u v : FourVelocity) : PreservesηLin (toMatrix u v)
```

def toLorentz[\[source\]](#)

A generalised boost as an element of the Lorentz Group.

```
def toLorentz (u v : FourVelocity) : LorentzGroup
```

lemma toLorentz_continuous[\[source\]](#)

```
lemma toLorentz_continuous (u : FourVelocity) : Continuous (toLorentz u)
```

lemma toLorentz_joined_to_1[\[source\]](#)

```
lemma toLorentz_joined_to_1 (u v : FourVelocity) : Joined 1 (toLorentz u v)
```

lemma toLorentz_in_connected_component_1[\[source\]](#)

```
lemma toLorentz_in_connected_component_1 (u v : FourVelocity) :
  toLorentz u v ∈ connectedComponent 1
```

lemma isProper[\[source\]](#)

```
lemma isProper (u v : FourVelocity) : IsProper (toLorentz u v)
```

2.29 HepLean.SpaceTime.LorentzGroup.Orthochronous

[HepLean/SpaceTime/LorentzGroup/Orthochronous.lean](#) on GitHub.

def fstCol[\[source\]](#)

The first column of a lorentz matrix as a PreFourVelocity.

```
@[simp]
def fstCol (Λ : LorentzGroup) : PreFourVelocity
```

def IsOrthochronous[\[source\]](#)

A Lorentz transformation is orthochronous if its $0\ 0$ element is non-negative.

```
def IsOrthochronous (Λ : LorentzGroup) : Prop
```

lemma IsOrthochronous_iff_transpose[\[source\]](#)

```
lemma IsOrthochronous_iff_transpose (Λ : LorentzGroup) :  
  IsOrthochronous Λ ↔ IsOrthochronous (transpose Λ)
```

lemma IsOrthochronous_iff_fstCol_IsFourVelocity[\[source\]](#)

```
lemma IsOrthochronous_iff_fstCol_IsFourVelocity (Λ : LorentzGroup) :  
  IsOrthochronous Λ ↔ IsFourVelocity (fstCol Λ)
```

def mapZeroZeroComp[\[source\]](#)

The continuous map taking a Lorentz transformation to its $0\ 0$ element.

```
def mapZeroZeroComp : C(LorentzGroup, ℝ)
```

def stepFunction[\[source\]](#)

An auxillary function used in the definition of orthchroMapReal.

```
def stepFunction : ℝ → ℝ
```

lemma stepFunction_continuous[\[source\]](#)

```
lemma stepFunction_continuous : Continuous stepFunction
```

def orthchroMapReal[\[source\]](#)

The continuous map from LorentzGroup to $\mathbb{R}$ wh taking Orthochronous elements to 1 and non-orthochronous to -1.

```
def orthchroMapReal : C(LorentzGroup, ℝ)
```

lemma orthchroMapReal_on_IsOrthochronous[\[source\]](#)

```
lemma orthchroMapReal_on_IsOrthochronous {Λ : LorentzGroup} (h : IsOrthochronous Λ) :  
  orthchroMapReal Λ = 1
```

lemma orthchroMapReal_on_not_IsOrthochronous[\[source\]](#)

```
lemma orthchroMapReal_on_not_IsOrthochronous {Λ : LorentzGroup} (h : ¬ IsOrthochronous Λ) :  
  orthchroMapReal Λ = - 1
```

lemma orthchroMapReal_minus_one_or_one[\[source\]](#)

```
lemma orthchroMapReal_minus_one_or_one (Λ : LorentzGroup) :
  orthchroMapReal Λ = -1 ∨ orthchroMapReal Λ = 1
```

def orthchroMap[\[source\]](#)

A continuous map from lorentzGroup to $\mathbb{Z}_2$ whose kernel are the Orthochronous elements.

```
def orthchroMap : C(LorentzGroup,  $\mathbb{Z}_2$ )
```

lemma orthchroMap_IsOrthochronous[\[source\]](#)

```
lemma orthchroMap_IsOrthochronous {Λ : LorentzGroup} (h : IsOrthochronous Λ) :
  orthchroMap Λ = 1
```

lemma orthchroMap_not_IsOrthochronous[\[source\]](#)

```
lemma orthchroMap_not_IsOrthochronous {Λ : LorentzGroup} (h : ¬ IsOrthochronous Λ) :
  orthchroMap Λ = Additive.toMul (1 : ZMod 2)
```

lemma zero_zero_mul[\[source\]](#)

```
lemma zero_zero_mul (Λ Λ' : LorentzGroup) :
  (Λ * Λ').1 0 0 = (fstCol (transpose Λ)).1 0 * (fstCol Λ').1 0 +
  ⟨⟨fstCol (transpose Λ)).1.space, (fstCol Λ').1.space⟩⟩_ℝ
```

lemma mul_othchron_of_othchron_othchron[\[source\]](#)

```
lemma mul_othchron_of_othchron_othchron {Λ Λ' : LorentzGroup} (h : IsOrthochronous Λ)
  (h' : IsOrthochronous Λ') : IsOrthochronous (Λ * Λ')
```

lemma mul_othchron_of_not_othchron_not_othchron[\[source\]](#)

```
lemma mul_othchron_of_not_othchron_not_othchron {Λ Λ' : LorentzGroup} (h : ¬ IsOrthochronous
  Λ)
  (h' : ¬ IsOrthochronous Λ') : IsOrthochronous (Λ * Λ')
```

lemma mul_not_othchron_of_othchron_not_othchron[\[source\]](#)

```
lemma mul_not_othchron_of_othchron_not_othchron {Λ Λ' : LorentzGroup} (h : IsOrthochronous Λ)
  (h' : ¬ IsOrthochronous Λ') : ¬ IsOrthochronous (Λ * Λ')
```

lemma mul_not_othchron_of_not_othchron_othchron[\[source\]](#)

```
lemma mul_not_othchron_of_not_othchron_othchron {Λ Λ' : LorentzGroup} (h : ¬ IsOrthochronous Λ)
  (h' : IsOrthochronous Λ') : ¬ IsOrthochronous (Λ * Λ')
```


def orthchroRep[\[source\]](#)

The homomorphism from `lorentzGroup` to $\mathbb{Z}_2$ whose kernel are the Orthochronous elements.

```
def orthchroRep : LorentzGroup →* ℤ₂ where
```

2.30 HepLean.SpaceTime.LorentzGroup.Proper

[HepLean/SpaceTime/LorentzGroup/Proper.lean](#) on GitHub.

lemma det_eq_one_or_neg_one[\[source\]](#)

The determinant of a member of the lorentz group is 1 or -1.

```
lemma det_eq_one_or_neg_one (Λ : ℒ) : Λ.1.det = 1 ∨ Λ.1.det = -1
```

instance TopologicalSpace ℤ₂[\[source\]](#)

```
instance : TopologicalSpace ℤ₂
```

instance DiscreteTopology ℤ₂[\[source\]](#)

```
instance : DiscreteTopology ℤ₂
```

instance TopologicalGroup ℤ₂[\[source\]](#)

```
instance : TopologicalGroup ℤ₂
```

def coeForℤ₂[\[source\]](#)

A continuous function from $(\{-1, 1\} : \text{Set } \mathbb{R})$ to $\mathbb{Z}_2$.

```
@[simps!]
def coeForℤ₂ : C(({-1, 1} : Set ℝ), ℤ₂) where
```

def detContinuous[\[source\]](#)

The continuous map taking a lorentz matrix to its determinant.

```
def detContinuous : C(ℒ, ℤ₂)
```

lemma detContinuous_eq_iff_det_eq[\[source\]](#)

```
lemma detContinuous_eq_iff_det_eq (Λ Λ' : LorentzGroup) :
  detContinuous Λ = detContinuous Λ' ↔ Λ.1.det = Λ'.1.det
```

def detRep[\[source\]](#)

The representation taking a lorentz matrix to its determinant.

```
@[simps!]
def detRep : ℒ →* ℤ₂ where
```

lemma detRep_continuous[\[source\]](#)

```
lemma detRep_continuous : Continuous detRep
```

lemma det_on_connected_component[\[source\]](#)

```
lemma det_on_connected_component {Λ Λ' : LorentzGroup} (h : Λ' ∈ connectedComponent Λ) :
  Λ.1.det = Λ'.1.det
```

lemma detRep_on_connected_component[\[source\]](#)

```
lemma detRep_on_connected_component {Λ Λ' : LorentzGroup} (h : Λ' ∈ connectedComponent Λ) :
  detRep Λ = detRep Λ'
```

lemma det_of_joined[\[source\]](#)

```
lemma det_of_joined {Λ Λ' : LorentzGroup} (h : Joined Λ Λ') : Λ.1.det = Λ'.1.det
```

def IsProper[\[source\]](#)

A Lorentz Matrix is proper if its determinant is 1.

```
@[simp]
```

```
def IsProper (Λ : LorentzGroup) : Prop
```

instance DecidablePred IsProper[\[source\]](#)

```
instance : DecidablePred IsProper
```

lemma IsProper_iff[\[source\]](#)

```
lemma IsProper_iff (Λ : LorentzGroup) : IsProper Λ ↔ detRep Λ = 1
```

lemma id_IsProper[\[source\]](#)

```
lemma id_IsProper : IsProper 1
```

lemma isProper_on_connected_component[\[source\]](#)

```
lemma isProper_on_connected_component {Λ Λ' : LorentzGroup} (h : Λ' ∈ connectedComponent Λ) :
  IsProper Λ ↔ IsProper Λ'
```

2.31 HepLean.SpaceTime.LorentzGroup.Rotations

[HepLean/SpaceTime/LorentzGroup/Rotations.lean](#) on GitHub.

def S03ToMatrix[\[source\]](#)

Given a element of $SO(3)$ the matrix corresponding to a space-time rotation.

```
@[simp]
```

```
def S03ToMatrix (A : SO(3)) : Matrix (Fin 4) (Fin 4) ℝ
```

lemma S03ToMatrix_PreservesηLin[\[source\]](#)

```
lemma S03ToMatrix_PreservesηLin (A : S0(3)) : PreservesηLin $ S03ToMatrix A
```

lemma S03ToMatrix_injective[\[source\]](#)

```
lemma S03ToMatrix_injective : Function.Injective S03ToMatrix
```

def S03ToLorentz[\[source\]](#)

Given a element of $S0(3)$ the element of the Lorentz group corresponding to a space-time rotation.

```
def S03ToLorentz : S0(3) →* ℒ where
```

2.32 HepLean.SpaceTime.Metric

[HepLean/SpaceTime/Metric.lean](#) on GitHub.

def η[\[source\]](#)

The metric as a 4×4 real matrix.

```
def η : Matrix (Fin 4) (Fin 4) ℝ
```

def ηInv[\[source\]](#)

The inverse of η . Used for notation.

```
def ηInv : Matrix (Fin 4) (Fin 4) ℝ
```

def ηUpDown[\[source\]](#)

η with $\hat{\eta}_{\mu\nu}$ indices. This is equivalent to the identity. This is used for notation.

```
def ηUpDown : Matrix (Fin 4) (Fin 4) ℝ
```

lemma η_block[\[source\]](#)

```
lemma η_block : η = Matrix.reindex finSumFinEquiv finSumFinEquiv (
  Matrix.fromBlocks (1 : Matrix (Fin 1) (Fin 1) ℝ) 0 0 (-1 : Matrix (Fin 3) (Fin 3) ℝ))
```

lemma η_reindex[\[source\]](#)

```
lemma η_reindex : (Matrix.reindex finSumFinEquiv finSumFinEquiv).symm η =
  LieAlgebra.Orthogonal.indefiniteDiagonal (Fin 1) (Fin 3) ℝ
```

lemma η_explicit[\[source\]](#)

```
lemma η_explicit : η = !![(1 : ℝ), 0, 0, 0; 0, -1, 0, 0; 0, 0, -1, 0; 0, 0, 0, -1]
```

lemma $\eta_{\text{off_diagonal}}$ [\[source\]](#)

```
@[simp]
lemma  $\eta_{\text{off\_diagonal}}$  { $\mu$   $\nu$  : Fin 4} (h :  $\mu \neq \nu$ ) :  $\eta \mu \nu = 0$ 
```

lemma $\eta_{\text{symmetric}}$ [\[source\]](#)

```
lemma  $\eta_{\text{symmetric}}$  ( $\mu$   $\nu$  : Fin 4) :  $\eta \mu \nu = \eta \nu \mu$ 
```

lemma $\eta_{\text{transpose}}$ [\[source\]](#)

```
@[simp]
lemma  $\eta_{\text{transpose}}$  :  $\eta.\text{transpose} = \eta$ 
```

lemma $\det_{\eta}$ [\[source\]](#)

```
@[simp]
lemma  $\det_{\eta}$  :  $\eta.\text{det} = -1$ 
```

lemma η_{sq} [\[source\]](#)

```
@[simp]
lemma  $\eta_{\text{sq}}$  :  $\eta * \eta = 1$ 
```

lemma $\eta_{\text{diag_mul_self}}$ [\[source\]](#)

```
lemma  $\eta_{\text{diag\_mul\_self}}$  ( $\mu$  : Fin 4) :  $\eta \mu \mu * \eta \mu \mu = 1$ 
```

lemma η_{mulVec} [\[source\]](#)

```
lemma  $\eta_{\text{mulVec}}$  (x : SpaceTime) :  $\eta *_v x = ![x_0, -x_1, -x_2, -x_3]$ 
```

lemma $\eta_{\text{as_diagonal}}$ [\[source\]](#)

```
lemma  $\eta_{\text{as\_diagonal}}$  :  $\eta = \text{diagonal } ![1, -1, -1, -1]$ 
```

lemma η_{mul} [\[source\]](#)

```
lemma  $\eta_{\text{mul}}$  ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) ( $\mu$   $\rho$  : Fin 4) :
   $[\eta * \Lambda]^{\mu}_{[\rho]} = [\eta]^{\mu}_{[\mu]} * [\Lambda]^{\mu}_{[\rho]}$ 
```

lemma mul_{η} [\[source\]](#)

```
lemma  $\text{mul}_{\eta}$  ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) ( $\mu$   $\rho$  : Fin 4) :
   $[\Lambda * \eta]^{\mu}_{[\rho]} = [\Lambda]^{\mu}_{[\mu]} * [\eta]^{\rho}_{[\rho]}$ 
```

lemma $\eta_{\text{mul_self}}$ [\[source\]](#)

```
lemma  $\eta_{\text{mul\_self}}$  ( $\mu$   $\nu$  : Fin 4) :  $\eta^{\nu}_{[\nu]} * \eta_{[\nu]}_{[\nu]} = \eta_{[\mu]}_{[\nu]}$ 
```

lemma $\eta_{\text{contract_self}}$ [\[source\]](#)

```
lemma  $\eta_{\text{contract\_self}}$  ( $\mu \nu : \text{Fin } 4$ ) :  $\sum x, (\eta[x]_{[\mu]} * \eta[x]_{[\nu]}) = \eta_{[\mu]}_{[\nu]}$ 
```

lemma $\eta_{\text{contract_self'}}$ [\[source\]](#)

```
lemma  $\eta_{\text{contract\_self'}}$  ( $\mu \nu : \text{Fin } 4$ ) :  $\sum x, (\eta[x]_{[\mu]} * \eta_{[\nu]}[x]) = \eta_{[\nu]}_{[\mu]}$ 
```

def linearMapForSpaceTime[\[source\]](#)

*Given a point in spaceTime x the linear map $y \rightarrow x \cdot_v (\eta *_{\nu} y)$.*

```
@[simps!]
```

```
def linearMapForSpaceTime ( $x : \text{SpaceTime}$ ) :  $\text{SpaceTime} \rightarrow_l [\mathbb{R}] \mathbb{R}$  where
```

def η_{Lin} [\[source\]](#)

The metric as a bilinear map from spaceTime to $\mathbb{R}$.

```
def  $\eta_{\text{Lin}}$  : LinearMap.BilinForm  $\mathbb{R}$  SpaceTime where
```

lemma $\eta_{\text{Lin_expand}}$ [\[source\]](#)

```
lemma  $\eta_{\text{Lin\_expand}}$  ( $x \ y : \text{SpaceTime}$ ) :  $\eta_{\text{Lin}} \ x \ y = x \ 0 * y \ 0 - x \ 1 * y \ 1 - x \ 2 * y \ 2 - x \ 3 * y \ 3$ 
```

lemma $\eta_{\text{Lin_expand_self}}$ [\[source\]](#)

```
lemma  $\eta_{\text{Lin\_expand\_self}}$  ( $x : \text{SpaceTime}$ ) :  $\eta_{\text{Lin}} \ x \ x = x \ 0^2 - \|x.\text{space}\|^2$ 
```

lemma time_elm_sq_of_ η_{Lin} [\[source\]](#)

```
lemma time_elm_sq_of_ $\eta_{\text{Lin}}$  ( $x : \text{SpaceTime}$ ) :  $x \ 0^2 = \eta_{\text{Lin}} \ x \ x + \|x.\text{space}\|^2$ 
```

lemma $\eta_{\text{Lin_leq_time_sq}}$ [\[source\]](#)

```
lemma  $\eta_{\text{Lin\_leq\_time\_sq}}$  ( $x : \text{SpaceTime}$ ) :  $\eta_{\text{Lin}} \ x \ x \leq x \ 0^2$ 
```

lemma $\eta_{\text{Lin_space_inner_product}}$ [\[source\]](#)

```
lemma  $\eta_{\text{Lin\_space\_inner\_product}}$  ( $x \ y : \text{SpaceTime}$ ) :  
   $\eta_{\text{Lin}} \ x \ y = x \ 0 * y \ 0 - \langle\langle x.\text{space}, y.\text{space} \rangle\rangle_{\mathbb{R}}$ 
```

lemma $\eta_{\text{Lin_ge_abs_inner_product}}$ [\[source\]](#)

```
lemma  $\eta_{\text{Lin\_ge\_abs\_inner\_product}}$  ( $x \ y : \text{SpaceTime}$ ) :  
   $x \ 0 * y \ 0 - \|\langle\langle x.\text{space}, y.\text{space} \rangle\rangle_{\mathbb{R}}\| \leq (\eta_{\text{Lin}} \ x \ y)$ 
```

lemma $\eta_{\text{Lin_ge_sub_norm}}$ [\[source\]](#)

```
lemma  $\eta_{\text{Lin\_ge\_sub\_norm}}$  ( $x \ y : \text{SpaceTime}$ ) :  
   $x \ 0 * y \ 0 - \|x.\text{space}\| * \|y.\text{space}\| \leq (\eta_{\text{Lin}} \ x \ y)$ 
```

lemma $\eta\text{Lin_symm}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_symm}$  (x y : SpaceTime) :  $\eta\text{Lin}$  x y =  $\eta\text{Lin}$  y x
```

lemma $\eta\text{Lin_stdBasis_apply}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_stdBasis\_apply}$  ( $\mu$  : Fin 4) (x : SpaceTime) :  $\eta\text{Lin}$  (stdBasis  $\mu$ ) x =  $\eta$   $\mu$   $\mu$  * x  $\mu$ 
```

lemma $\eta\text{Lin_}\eta\text{_stdBasis}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_}\eta\text{\_stdBasis}$  ( $\mu$  v : Fin 4) :  $\eta\text{Lin}$  (stdBasis  $\mu$ ) (stdBasis v) =  $\eta$   $\mu$  v
```

lemma $\eta\text{Lin_mulVec_left}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_mulVec\_left}$  (x y : SpaceTime) ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :  
   $\eta\text{Lin}$  ( $\Lambda$  *v x) y =  $\eta\text{Lin}$  x (( $\eta$  *  $\Lambda^T$  *  $\eta$ ) *v y)
```

lemma $\eta\text{Lin_mulVec_right}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_mulVec\_right}$  (x y : SpaceTime) ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :  
   $\eta\text{Lin}$  x ( $\Lambda$  *v y) =  $\eta\text{Lin}$  (( $\eta$  *  $\Lambda^T$  *  $\eta$ ) *v x) y
```

lemma $\eta\text{Lin_matrix_stdBasis}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_matrix\_stdBasis}$  ( $\mu$  v : Fin 4) ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :  
   $\eta\text{Lin}$  (stdBasis v) ( $\Lambda$  *v stdBasis  $\mu$ ) =  $\eta$  v v *  $\Lambda$  v  $\mu$ 
```

lemma $\eta\text{Lin_matrix_stdBasis}'$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_matrix\_stdBasis}'$  ( $\mu$  v : Fin 4) ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :  
   $\Lambda$  v  $\mu$  =  $\eta$  v v *  $\eta\text{Lin}$  (stdBasis v) ( $\Lambda$  *v stdBasis  $\mu$ )
```

lemma $\eta\text{Lin_matrix_eq_identity_iff}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_matrix\_eq\_identity\_iff}$  ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :  
   $\Lambda$  = 1  $\leftrightarrow$   $\forall$  (x y : SpaceTime),  $\eta\text{Lin}$  x y =  $\eta\text{Lin}$  x ( $\Lambda$  *v y)
```

def quadraticForm[\[source\]](#)

The metric as a quadratic form on spaceTime.

```
def quadraticForm : QuadraticForm  $\mathbb{R}$  SpaceTime
```

2.33 HepLean.SpaceTime.SL2C.Basic

[HepLean/SpaceTime/SL2C/Basic.lean](#) on GitHub.

def toLinearMapSelfAdjointMatrix[\[source\]](#)

*Given an element $M \in SL(2, \mathbb{C})$ the linear map from selfAdjoint (Matrix (Fin 2) (Fin 2) $\mathbb{C}$) to itself defined by $A \mapsto M * A * M^H$.*

```
@[simps!]
def toLinearMapSelfAdjointMatrix (M : SL(2, ℂ)) :
  selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ) →ℓ[ℝ] selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ) where
```

def repSelfAdjointMatrix [\[source\]](#)

*The representation of $SL(2, \mathbb{C})$ on $\text{selfAdjoint (Matrix (Fin 2) (Fin 2) } \mathbb{C})$ given by $M \cdot A \mapsto M * A * M^H$.*

```
@[simps!]
def repSelfAdjointMatrix : Representation ℝ SL(2, ℂ) $ selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)
  where
```

def repSpaceTime [\[source\]](#)

The representation of $SL(2, \mathbb{C})$ on spaceTime obtained from $\text{toSelfAdjointMatrix}$ and $\text{repSelfAdjointMatrix}$.

```
def repSpaceTime : Representation ℝ SL(2, ℂ) SpaceTime where
```

def toLorentzGroupElem [\[source\]](#)

Given an element $M \in SL(2, \mathbb{C})$ the corresponding element of the Lorentz group.

```
@[simps!]
def toLorentzGroupElem (M : SL(2, ℂ)) : ℒ
```

def toLorentzGroup [\[source\]](#)

The group homomorphism from $SL(2, \mathbb{C})$ to the Lorentz group $\mathcal{L}$.

```
@[simps!]
def toLorentzGroup : SL(2, ℂ) →* ℒ where
```

2.34 HepLean.StandardModel.Basic

[HepLean/StandardModel/Basic.lean](#) on GitHub.

abbrev GaugeGroup [\[source\]](#)

The global gauge group of the standard model. TODO: Generalize to quotient.

```
abbrev GaugeGroup : Type
```

2.35 HepLean.StandardModel.HiggsBoson.Basic

[HepLean/StandardModel/HiggsBoson/Basic.lean](#) on GitHub.

abbrev HiggsBundle [\[source\]](#)

The trivial vector bundle $\mathcal{R}^2 \times \mathbb{C}^2$. (TODO: Make associated bundle.)

```
abbrev HiggsBundle
```

instance SmoothVectorBundle HiggsVec HiggsBundle SpaceTime.asSmoothManifold [\[source\]](#)

instance : SmoothVectorBundle HiggsVec HiggsBundle SpaceTime.asSmoothManifold

abbrev HiggsField [\[source\]](#)

A higgs field is a smooth section of the higgs bundle.

abbrev HiggsField : Type

def toField [\[source\]](#)

Given a vector $\mathbb{C}^2$ the constant higgs field with value equal to that section.

noncomputable def HiggsVec.toField (ϕ : HiggsVec) : HiggsField **where**

def toHiggsVec [\[source\]](#)

Given a higgsField, the corresponding map from spaceTime to higgsVec.

def toHiggsVec (ϕ : HiggsField) : SpaceTime $\rightarrow$ HiggsVec

lemma toHiggsVec_smooth [\[source\]](#)

lemma toHiggsVec_smooth (ϕ : HiggsField) : Smooth $\mathcal{I}(\mathbb{R}, \text{SpaceTime}) \mathcal{I}(\mathbb{R}, \text{HiggsVec}) \phi.\text{toHiggsVec}$

lemma toField_toHiggsVec_apply [\[source\]](#)

lemma toField_toHiggsVec_apply (ϕ : HiggsField) (x : SpaceTime) :
($\phi.\text{toHiggsVec } x$).toField $x = \phi x$

lemma higgsVecToFin2C_toHiggsVec [\[source\]](#)

lemma higgsVecToFin2C_toHiggsVec (ϕ : HiggsField) :
higgsVecToFin2C $\circ \phi.\text{toHiggsVec} = \phi$

lemma toVec_smooth [\[source\]](#)

lemma toVec_smooth (ϕ : HiggsField) : Smooth $\mathcal{I}(\mathbb{R}, \text{SpaceTime}) \mathcal{I}(\mathbb{R}, \text{Fin } 2 \rightarrow \mathbb{C}) \phi$

lemma apply_smooth [\[source\]](#)

lemma apply_smooth (ϕ : HiggsField) :
 $\forall i, \text{Smooth } \mathcal{I}(\mathbb{R}, \text{SpaceTime}) \mathcal{I}(\mathbb{R}, \mathbb{C})$ (**fun** (x : SpaceTime) $\Rightarrow (\phi x i)$)

lemma apply_re_smooth [\[source\]](#)

lemma apply_re_smooth (ϕ : HiggsField) (i : Fin 2):
Smooth $\mathcal{I}(\mathbb{R}, \text{SpaceTime}) \mathcal{I}(\mathbb{R}, \mathbb{R})$ (reCLM $\circ$ (**fun** (x : SpaceTime) $\Rightarrow (\phi x i)$))

lemma apply_im_smooth[\[source\]](#)

```
lemma apply_im_smooth (φ : HiggsField) (i : Fin 2):
  Smooth  $\mathcal{I}(\mathbb{R}, \text{SpaceTime}) \mathcal{I}(\mathbb{R}, \mathbb{R})$  (imCLM ∘ (fun (x : SpaceTime) => (φ x i)))
```

def innerProd[\[source\]](#)

Given two higgsField, the map $\text{spaceTime} \rightarrow \mathbb{C}$ obtained by taking their inner product.

```
def innerProd (φ1 φ2 : HiggsField) : SpaceTime → ℂ
```

def normSq[\[source\]](#)

Given a higgsField, the map $\text{spaceTime} \rightarrow \mathbb{R}$ obtained by taking the square norm of the higgs vector.

```
@[simp]
def normSq (φ : HiggsField) : SpaceTime → ℝ
```

lemma toHiggsVec_norm[\[source\]](#)

```
lemma toHiggsVec_norm (φ : HiggsField) (x : SpaceTime) :
  ||φ x|| = ||φ.toHiggsVec x||
```

lemma normSq_expand[\[source\]](#)

```
lemma normSq_expand (φ : HiggsField) :
  φ.normSq = fun x => (conj (φ x 0) * (φ x 0) + conj (φ x 1) * (φ x 1)).re
```

lemma normSq_smooth[\[source\]](#)

```
lemma normSq_smooth (φ : HiggsField) : Smooth  $\mathcal{I}(\mathbb{R}, \text{SpaceTime}) \mathcal{I}(\mathbb{R}, \mathbb{R})$  φ.normSq
```

lemma normSq_nonneg[\[source\]](#)

```
lemma normSq_nonneg (φ : HiggsField) (x : SpaceTime) : 0 ≤ φ.normSq x
```

lemma normSq_zero[\[source\]](#)

```
lemma normSq_zero (φ : HiggsField) (x : SpaceTime) : φ.normSq x = 0 ↔ φ x = 0
```

def potential[\[source\]](#)

*The Higgs potential of the form $-\mu^2 * |\varphi|^2 + \lambda * |\varphi|^4$.*

```
@[simp]
def potential (φ : HiggsField) (μSq lambda : ℝ) (x : SpaceTime) : ℝ
```

lemma potential_smooth[\[source\]](#)

```
lemma potential_smooth (φ : HiggsField) (μSq lambda : ℝ) :
  Smooth  $\mathcal{I}(\mathbb{R}, \text{SpaceTime}) \mathcal{I}(\mathbb{R}, \mathbb{R})$  (fun x => φ.potential μSq lambda x)
```

lemma potential_apply[\[source\]](#)

```
lemma potential_apply (φ : HiggsField) (μSq lambda : ℝ) (x : SpaceTime) :
  (φ.potential μSq lambda) x = HiggsVec.potential μSq lambda (φ.toHiggsVec x)
```

def IsConst[\[source\]](#)

A higgs field is constant if it is equal for all x y in spaceTime.

```
def IsConst (Φ : HiggsField) : Prop
```

lemma isConst_of_higgsVec[\[source\]](#)

```
lemma isConst_of_higgsVec (φ : HiggsVec) : φ.toField.IsConst
```

lemma isConst_iff_of_higgsVec[\[source\]](#)

```
lemma isConst_iff_of_higgsVec (Φ : HiggsField) : Φ.IsConst ↔ ∃ (φ : HiggsVec), Φ = φ.toField
```

lemma normSq_of_higgsVec[\[source\]](#)

```
lemma normSq_of_higgsVec (φ : HiggsVec) : φ.toField.normSq = fun x => (norm φ) ^ 2
```

lemma potential_of_higgsVec[\[source\]](#)

```
lemma potential_of_higgsVec (φ : HiggsVec) (μSq lambda : ℝ) :
  φ.toField.potential μSq lambda = fun _ => HiggsVec.potential μSq lambda φ
```

2.36 HepLean.StandardModel.HiggsBoson.TargetSpace

[HepLean/StandardModel/HiggsBoson/TargetSpace.lean](#) on GitHub.

abbrev HiggsVec[\[source\]](#)

The complex vector space in which the Higgs field takes values.

```
abbrev HiggsVec
```

def higgsRepUnitary[\[source\]](#)

The Higgs representation as a homomorphism from the gauge group to unitary 2×2-matrices.

```
@[simps!]
noncomputable def higgsRepUnitary : GaugeGroup →* unitaryGroup (Fin 2) ℂ where
```

def orthonormBasis[\[source\]](#)

An orthonormal basis of higgsVec.

```
noncomputable def orthonormBasis : OrthonormalBasis (Fin 2) ℂ HiggsVec
```

def matrixToLin[\[source\]](#)

Takes in a 2×2 -matrix and returns a linear map of `higgsVec`.

```
noncomputable def matrixToLin : Matrix (Fin 2) (Fin 2) ℂ →* (HiggsVec →L[ℂ] HiggsVec) where
```

lemma matrixToLin_star[\[source\]](#)

```
lemma matrixToLin_star (g : Matrix (Fin 2) (Fin 2) ℂ) :  
  matrixToLin (star g) = star (matrixToLin g)
```

lemma matrixToLin_unitary[\[source\]](#)

```
lemma matrixToLin_unitary (g : unitaryGroup (Fin 2) ℂ) :  
  matrixToLin g ∈ unitary (HiggsVec →L[ℂ] HiggsVec)
```

def unitaryToLin[\[source\]](#)

The natural homomorphism from unitary 2×2 complex matrices to unitary transformations of `higgsVec`.

```
noncomputable def unitaryToLin : unitaryGroup (Fin 2) ℂ →* unitary (HiggsVec →L[ℂ] HiggsVec)  
  where
```

def unitToLinear[\[source\]](#)

The inclusion of unitary transformations on `higgsVec` into all linear transformations.

```
@[simps!]  
def unitToLinear : unitary (HiggsVec →L[ℂ] HiggsVec) →* HiggsVec →L[ℂ] HiggsVec
```

def rep[\[source\]](#)

The representation of the gauge group acting on `higgsVec`.

```
@[simps!]  
def rep : Representation ℂ GaugeGroup HiggsVec
```

lemma higgsRepUnitary_mul[\[source\]](#)

```
lemma higgsRepUnitary_mul (g : GaugeGroup) (φ : HiggsVec) :  
  (higgsRepUnitary g).1 *v φ = g.2.2 ^ 3 • (g.2.1.1 *v φ)
```

lemma rep_apply[\[source\]](#)

```
lemma rep_apply (g : GaugeGroup) (φ : HiggsVec) : rep g φ = g.2.2 ^ 3 • (g.2.1.1 *v φ)
```

lemma norm_invariant[\[source\]](#)

```
lemma norm_invariant (g : GaugeGroup) (φ : HiggsVec) : ||rep g φ|| = ||φ||
```

def potential[\[source\]](#)

The higgs potential for higgsVec, i.e. for constant higgs fields.

```
def potential (φ : HiggsVec) : ℝ
```

lemma potential_invariant[\[source\]](#)

```
lemma potential_invariant (φ : HiggsVec) (g : GaugeGroup) :
  potential μSq (λ) (rep g φ) = potential μSq (λ) φ
```

lemma potential_as_quad[\[source\]](#)

```
lemma potential_as_quad (φ : HiggsVec) :
  λ * ||φ|| ^ 2 * ||φ|| ^ 2 + (- μSq ) * ||φ|| ^ 2 + (- potential μSq (λ) φ) = 0
```

lemma potential_snd_term_nonneg[\[source\]](#)

```
lemma potential_snd_term_nonneg (φ : HiggsVec) :
  0 ≤ λ * ||φ|| ^ 4
```

lemma zero_le_potential_discrim[\[source\]](#)

```
lemma zero_le_potential_discrim (φ : HiggsVec) :
  0 ≤ discrim (λ) (- μSq ) (- potential μSq (λ) φ)
```

lemma potential_eq_zero_sol[\[source\]](#)

```
lemma potential_eq_zero_sol (φ : HiggsVec)
  (hV : potential μSq (λ) φ = 0) : φ = 0 ∨ ||φ|| ^ 2 = μSq / λ
```

lemma potential_eq_zero_sol_of_μSq_nonpos[\[source\]](#)

```
lemma potential_eq_zero_sol_of_μSq_nonpos (hμSq : μSq ≤ 0)
  (φ : HiggsVec) (hV : potential μSq (λ) φ = 0) : φ = 0
```

lemma potential_bounded_below[\[source\]](#)

```
lemma potential_bounded_below (φ : HiggsVec) :
  - μSq ^ 2 / (4 * (λ)) ≤ potential μSq (λ) φ
```

lemma potential_bounded_below_of_μSq_nonpos[\[source\]](#)

```
lemma potential_bounded_below_of_μSq_nonpos {μSq : ℝ}
  (hμSq : μSq ≤ 0) (φ : HiggsVec) : 0 ≤ potential μSq (λ) φ
```

lemma potential_eq_bound_discrim_zero[\[source\]](#)

```
lemma potential_eq_bound_discrim_zero (φ : HiggsVec)
  (hV : potential μSq (λ) φ = - μSq ^ 2 / (4 * λ)) :
  discrim (λ) (- μSq) (- potential μSq (λ) φ) = 0
```

lemma potential_eq_bound_higgsVec_sq[\[source\]](#)

```
lemma potential_eq_bound_higgsVec_sq (φ : HiggsVec)
  (hV : potential μSq (λ) φ = - μSq ^ 2 / (4 * (λ))) :
  ||φ|| ^ 2 = μSq / (2 * λ)
```

lemma potential_eq_bound_iff[\[source\]](#)

```
lemma potential_eq_bound_iff (φ : HiggsVec) :
  potential μSq (λ) φ = - μSq ^ 2 / (4 * (λ)) ↔ ||φ|| ^ 2 = μSq / (2 * (λ))
```

lemma potential_eq_bound_iff_of_μSq_nonpos[\[source\]](#)

```
lemma potential_eq_bound_iff_of_μSq_nonpos {μSq : ℝ}
  (hμSq : μSq ≤ 0) (φ : HiggsVec) : potential μSq (λ) φ = 0 ↔ φ = 0
```

lemma potential_eq_bound_IsMin0n[\[source\]](#)

```
lemma potential_eq_bound_IsMin0n (φ : HiggsVec)
  (hV : potential μSq lambda φ = - μSq ^ 2 / (4 * lambda)) :
  IsMin0n (potential μSq lambda) Set.univ φ
```

lemma potential_eq_bound_IsMin0n_of_μSq_nonpos[\[source\]](#)

```
lemma potential_eq_bound_IsMin0n_of_μSq_nonpos {μSq : ℝ}
  (hμSq : μSq ≤ 0) (φ : HiggsVec) (hV : potential μSq lambda φ = 0) :
  IsMin0n (potential μSq lambda) Set.univ φ
```

lemma potential_bound_reached_of_μSq_nonneg[\[source\]](#)

```
lemma potential_bound_reached_of_μSq_nonneg {μSq : ℝ} (hμSq : 0 ≤ μSq) :
  ∃ (φ : HiggsVec), potential μSq lambda φ = - μSq ^ 2 / (4 * lambda)
```

lemma IsMin0n_potential_iff_of_μSq_nonneg[\[source\]](#)

```
lemma IsMin0n_potential_iff_of_μSq_nonneg {μSq : ℝ} (hμSq : 0 ≤ μSq) :
  IsMin0n (potential μSq lambda) Set.univ φ ↔ ||φ|| ^ 2 = μSq / (2 * lambda)
```

lemma IsMin0n_potential_iff_of_μSq_nonpos[\[source\]](#)

```
lemma IsMin0n_potential_iff_of_μSq_nonpos {μSq : ℝ} (hμSq : μSq ≤ 0) :
  IsMin0n (potential μSq lambda) Set.univ φ ↔ φ = 0
```

def rotateMatrix[\[source\]](#)

Given a Higgs vector, a rotation matrix which puts the first component of the vector to zero, and the second component to a real

```
def rotateMatrix (φ : HiggsVec) : Matrix (Fin 2) (Fin 2) ℂ
```

lemma rotateMatrix_star[\[source\]](#)

```
lemma rotateMatrix_star (φ : HiggsVec) :
  star φ.rotateMatrix =
    ![![conj (φ 1) / ||φ|| , φ 0 / ||φ||], ![ - conj (φ 0) / ||φ|| , φ 1 / ||φ|| ] ]
```

lemma rotateMatrix_det[\[source\]](#)

```
lemma rotateMatrix_det {φ : HiggsVec} (hφ : φ ≠ 0) : (rotateMatrix φ).det = 1
```

lemma rotateMatrix_unitary[\[source\]](#)

```
lemma rotateMatrix_unitary {φ : HiggsVec} (hφ : φ ≠ 0) :
  (rotateMatrix φ) ∈ unitaryGroup (Fin 2) ℂ
```

lemma rotateMatrix_specialUnitary[\[source\]](#)

```
lemma rotateMatrix_specialUnitary {φ : HiggsVec} (hφ : φ ≠ 0) :
  (rotateMatrix φ) ∈ specialUnitaryGroup (Fin 2) ℂ
```

def rotateGuageGroup[\[source\]](#)

Given a Higgs vector, an element of the gauge group which puts the first component of the vector to zero, and the second component to a real

```
def rotateGuageGroup {φ : HiggsVec} (hφ : φ ≠ 0) : GaugeGroup
```

lemma rotateGuageGroup_apply[\[source\]](#)

```
lemma rotateGuageGroup_apply {φ : HiggsVec} (hφ : φ ≠ 0) :
  rep (rotateGuageGroup hφ) φ = ![0, ofReal ||φ||]
```

theorem rotate_fst_zero_snd_real[\[source\]](#)

```
theorem rotate_fst_zero_snd_real (φ : HiggsVec) :
  ∃ (g : GaugeGroup), rep g φ = ![0, ofReal ||φ||]
```

2.37 scripts.check_file_imports

[scripts/check_file_imports.lean](#) on GitHub.

def addModulesIn[\[source\]](#)

Recursively finds files in directory.

```
partial def addModulesIn (recurse : Bool) (prev : Array Name) (root : Name := .anonymous)
  (path : FilePath) : IO (Array Name)
```

def expectedHepLeanImports[\[source\]](#)

Compute imports expected by HepLean.lean by looking at file structure.

```
def expectedHepLeanImports : IO (Array Name)
```

def arrayImportSorted[\[source\]](#)

Checks whether an array imports is sorted after Init is removed.

```
def arrayImportSorted (imports : Array Import) : IO Bool
```

def checkMissingImports[\[source\]](#)

Checks every file in reqImports is imported into modData return true if this is NOT the case.

```
def checkMissingImports (modData : ModuleData) (reqImports : Array Name) :  
  IO Bool
```

def main[\[source\]](#)

```
def main (_ : List String) : IO UInt32
```

2.38 scripts.type_former_lint

[scripts/type_former_lint.lean](#) on GitHub.

def IsUpperCamal[\[source\]](#)

A rough definition of Upper Camal, checking that if a string starts with a latin letter then that letter is captial.

```
def IsUpperCamal (s : String) : Bool
```

def main[\[source\]](#)

```
def main (_ : List String) : IO UInt32
```